INSTITUTO TECNOLÓGICO DE AERONÁUTICA



Marcelo Loiola Lopes Veras

EXPLORING DISTRIBUTED COMPUTING ARCHITECTURES FOR AEROSPACE SIMULATION ENVIRONMENTS

Final Paper
2026

Course of Computer Engineering

CDU CDU

Marcelo Loiola Lopes Veras

EXPLORING DISTRIBUTED COMPUTING ARCHITECTURES FOR AEROSPACE SIMULATION ENVIRONMENTS

Advisor

Prof. Dr. Juliana de Melo Bezerra (ITA)

Co-advisor

1° Ten. Eng. Adrisson Rogério Samersla (IEAv)

COMPUTER ENGINEERING

São José dos Campos
Instituto Tecnológico de Aeronáutica

2026

Cataloging-in Publication Data
Documentation and Information Division

Veras, Marcelo Loiola Lopes
Exploring Distributed Computing Architectures for Aerospace Simulation Environments / Marcelo Loiola Lopes Veras.
São José dos Campos, 2026.
92p.

Final paper (Undergraduation study) – Course of Computer Engineering– Instituto Tecnológico de Aeronáutica, 2026. Advisor: Prof. Dr. Juliana de Melo Bezerra. Co-advisor: 1° Ten. Eng. Adrisson Rogério Samersla.

I. Instituto Tecnológico de Aeronáutica. II. Exploring Distributed Computing Architectures for Aerospace Simulation Environments.

BIBLIOGRAPHIC REFERENCE

VERAS, Marcelo Loiola Lopes. **Exploring Distributed Computing Architectures for Aerospace Simulation Environments**. 2026. 92p. Final paper (Undergraduation study) – Instituto Tecnológico de Aeronáutica, São José dos Campos.

CESSION OF RIGHTS

AUTHOR'S NAME: Marcelo Loiola Lopes Veras
PUBLICATION TITLE: Exploring Distributed Computing Architectures for Aerospace Simulation Environments.
PUBLICATION KIND/YEAR: Final paper (Undergraduation study) / 2026

It is granted to Instituto Tecnológico de Aeronáutica permission to reproduce copies of this final paper and to only loan or to sell copies for academic and scientific purposes. The author reserves other publication rights and no part of this final paper can be reproduced without the authorization of the author.

Marcelo Loiola Lopes Veras
Rua H8D, Ap. 205
12.228-462 – São José dos Campos–SP

EXPLORING DISTRIBUTED COMPUTING ARCHITECTURES FOR AEROSPACE SIMULATION ENVIRONMENTS

This publication was accepted like Final Work of Undergraduation Study

---

Marcelo Loiola Lopes Veras

Author

Juliana de Melo Bezerra (ITA)

Advisor

Adrisson Rogério Samersla (IEAv)

Co-advisor

Prof. Dr. Marcos Ricardo Omena de Albuquerque Máximo
Course Coordinator of Computer Engineering

São José dos Campos: november 13, 2026.

Resumo

Ambientes de simulação aeroespacial em larga escala geram volumes substanciais de dados brutos cujo pós-processamento representa um gargalo crítico nos fluxos de trabalho analíticos. Quando lotes de simulação são processados sequencialmente em uma única máquina, o tempo necessário para derivar métricas significativas e valores agregados cresce proporcionalmente ao número de execuções, limitando a agilidade analítica necessária para o Planejamento Baseado em Capacidades em contextos de defesa.

Este trabalho propõe, implementa e avalia uma infraestrutura de computação distribuída projetada para acelerar o estágio de pós-processamento de simulações aeroespaciais baseadas em cenários, produzidas pelo projeto ASA, desenvolvido no Instituto de Estudos Avançados da Força Aérea Brasileira. O projeto, intitulado AeroSOuDA, combina o Apache Spark (via PySpark) para extração paralela de métricas com o Hadoop Distributed File System (HDFS) para armazenamento escalável com localidade de dados, expondo uma interface de linha de comando interativa por meio da qual analistas definem funções de métrica e de agregação antes do envio do trabalho, sem exigir conhecimentos computacionais elevados.

Dois modos de implantação são apresentados: um cluster local via Docker Compose para prototipagem, e uma implantação em máquina virtual remota acessada por um cliente Python via SSH e SFTP. A avaliação experimental em dois cenários analíticos sobre dois conjuntos de dados independentes (1,3 GB e 1,5 GB, respectivamente) demonstra speedups de 2,86× e 3,14× com quatro executores em um cluster de host único, correspondendo a eficiências paralelas de 71,4% e 78,6%. Um experimento de escalabilidade com um, dois e quatro executores no conjunto maior confirma que a arquitetura escala beneficamente, atingindo 93,4% de eficiência com dois executores. O overhead de comunicação SSH permanece inferior a 1% do tempo total de execução em ambas as campanhas, confirmando a viabilidade do modelo de implantação remota.

Os resultados estabelecem que o paradigma MapReduce, implementado via PySpark e HDFS, fornece uma base prática e escalável para o pós-processamento distribuído de lotes de simulação em um contexto aeroespacial de defesa, reduzindo o tempo de execução em 65–68% em relação ao referencial sequencial, mantendo o conjunto de dados

bruto integralmente no servidor. O speedup medido cresce com o tamanho do conjunto de dados, consistente com as previsões da Lei de Amdahl derivadas do experimento de escalabilidade.

Abstract

Large-scale aerospace simulation environments generate substantial volumes of raw data whose post-processing represents a critical bottleneck in analytical workflows. When simulation batches are processed sequentially on a single machine, the time required to derive meaningful metrics and aggregate values grows proportionally to the number of executions, limiting the analytical agility required for Capability-Based Planning in defense contexts.

This work proposes, implements, and evaluates a distributed computing infrastructure designed to accelerate the post-processing stage of scenario-based aerospace simulations produced by the ASA project, developed at the Institute for Advanced Studies of the Brazilian Air Force. The project, titled AeroSOuDA, combines Apache Spark (via PySpark) for parallel metric extraction with the Hadoop Distributed File System (HDFS) for scalable storage with data locality, exposing an interactive command-line interface through which analysts define metric and aggregation functions before submitting the job, without requiring advanced computational knowledge.

Two deployment modes are presented: a local cluster via Docker Compose for prototyping, and a remote virtual machine deployment accessed by a Python client over SSH and SFTP. The experimental evaluation in two analytical scenarios over two independent datasets (1,3 GB and 1,5 GB, respectively) demonstrates speedups of 2,86× and 3,14× with four executors in a single-host cluster, corresponding to parallel efficiencies of 71,4% and 78,6%. A scalability experiment with one, two, and four executors on the larger dataset confirms that the architecture scales beneficially, reaching 93,4% of efficiency with two executors. The SSH communication overhead remains below 1% of the total execution time in both campaigns, confirming the feasibility of the remote deployment model.

The results establish that the MapReduce paradigm, implemented via PySpark and HDFS, provides a practical and scalable foundation for the distributed post-processing of simulation batches in an aerospace defense context, reducing execution time by 65–68% relative to the sequential baseline while keeping the raw dataset entirely on the server. The measured speedup increases with the size of the dataset, consistent with the predictions of Amdahl's Law derived from the scalability experiment.

List of Figures

FIGURE 2.1 – Example of a defensive scenario modeled via Webstation. 22

FIGURE 2.2 – General Pipeline for Modeling and Simulation (M&S), showing both stages: Execution Stage and Analysis and Aggregation Stage. 24

FIGURE 2.3 – MapReduce model overview. 25

FIGURE 2.4 – Distributed memory overview. 32

FIGURE 3.1 – Current simulation-processing pipeline. 37

FIGURE 3.2 – Proposed simulation-processing pipeline. 39

FIGURE 4.1 – Execution pipeline of the Command-Line Interface (CLI) module (`cli.py`). 48

FIGURE 4.2 – Distributed processing pipeline of the CSV processor. 53

FIGURE 4.3 – AeroSOuDA local cluster data-flow diagram. 54

FIGURE 5.1 – Two-component architecture of the AeroSOuDA remote deployment. 63

FIGURE 5.2 – Execution pipeline of the AeroSOuDA remote client (`client/cli.py`). 68

FIGURE 5.3 – End-to-end data flow of the AeroSOuDA remote deployment. 68

FIGURE 6.1 – Benchmark measurement protocol. 74

List of Tables

TABLE 2.1 – Comparison of related works: processing paradigm, data access, and storage (part 1). 34

TABLE 2.2 – Comparison of related works: user interface, primary goal, and simulation domain (part 2). 35

TABLE 4.1 – Scenario YAML Ain't Markup Language (YAML) field reference for `x_variable` and `y_variable`. 43

TABLE 6.1 – Java Virtual Machine (JVM) warm-up check: run 1 vs. mean of runs 2–10. 76

TABLE 6.2 – Quick-Reaction, per-run timings (serial mode, 1 executor). 76

TABLE 6.3 – Quick-Reaction, per-run timings (parallel mode, 4 executors). 77

TABLE 6.4 – Quick-Reaction, summary statistics (all 10 runs per mode). 77

TABLE 6.5 – Launcher Longitude, per-run timings (serial mode, 1 executor). . . . 78

TABLE 6.6 – Launcher Longitude, per-run timings (parallel mode, 4 executors). . 78

TABLE 6.7 – Launcher Longitude, summary statistics (all 10 runs per mode). . . 79

TABLE 6.8 – Speedup and efficiency comparison across the two benchmark campaigns. 79

TABLE 6.9 – Throughput comparison across both benchmark campaigns (all 10 runs per mode). 80

TABLE 6.10 – Amdahl's law predictions vs. measured speedup for the scalability experiment (Launcher Longitude dataset). 82

TABLE 6.11 – Scalability experiment, mean T_{job} and speedup by executor count (5 runs per configuration, Launcher Longitude dataset). 82

List of Abbreviations and Acronyms

A&A	Analysis and Aggregation.
AeroSOuDA	Aerospace Simulation Output Distributed Architecture.
API	Application Programming Interface.
ASA	Aerospace Simulation Ambient.
AWS	Amazon Web Services.
CBP	Capabilities-Based Planning.
CFD	Computational Fluid Dynamics.
CLI	Command-Line Interface.
CPU	Central Processing Unit.
CSV	Comma-Separated Values.
DEM	Discrete Element Method.
DFS	Distributed File System.
DOE	Design of Experiments.
GBAD	Ground-Based Air Defence.
HDFS	Hadoop Distributed File System.
HPC	High-Performance Computing.
IP	Internet Protocol.
JIT	Just-In-Time.
JSON	JavaScript Object Notation.
JVM	Java Virtual Machine.
KVM	Kernel-based Virtual Machine.
M&S	Modeling and Simulation.

QR	Quick-Reaction.
RDD	Resilient Distributed Dataset.
RPC	Remote Procedure Call.
SA	Sensitivity Analysis.
SFTP	Secure File Transfer Protocol.
SQL	Structured Query Language.
SSH	Secure Shell.
TCP	Transmission Control Protocol.
TTY	Teletypewriter (terminal device).
UDF	User-Defined Function.
UI	User Interface.
URI	Uniform Resource Identifier.
VM	Virtual Machine.
VPN	Virtual Private Network.
YAML	YAML Ain't Markup Language.

Contents

1 Introduction 15

1.1 Contextualization 15

1.2 Motivation 16

1.3 Objective 17

2 Background 19

2.1 The ASA Project 19

2.2 Capacity-Based Planning (CBP) 19

2.3 Modeling and Simulation (M&S) 20

2.4 Sensitivity Analysis (SA) 21

2.5 Distributed Computing 23

2.5.1 Distribution Viability in a Simulation Workflow 23

2.5.2 MapReduce 25

2.5.3 Apache Spark 26

2.5.4 Virtual machines 28

2.5.5 SSH and SFTP 29

2.5.6 PySpark 30

2.5.7 Docker and Containerization 30

2.6 Distributed File Systems (DFS) 31

2.7 Related Work 33

3 Aerospace Simulation Output Distributed Architecture (AeroSOuDA) 37

3.1 Current Architecture 37

3.2 Proposed Architecture . . . 38

4 Local Cluster Implementation . . . 40

4.1 Cluster Infrastructure . . . 40

4.1.1 Container Orchestration with Docker Compose . . . 40

4.1.2 Scenario Configuration Format . . . 41

4.2 The Command-Line Interface Module . . . 47

4.2.1 Overview . . . 47

4.2.2 Pipeline Flow . . . 47

4.3 The Distributed CSV Processor . . . 50

4.3.1 Overview . . . 50

4.3.2 Scenario Configuration Resolution . . . 51

4.3.3 Spark Ingestion and Validation . . . 51

4.3.4 Distributed Map Phase . . . 51

4.3.5 Reduce Phase and Output Generation . . . 52

4.4 Module Integration . . . 54

4.5 HDFS Integration . . . 55

4.5.1 The Storage–Compute Separation Principle . . . 55

4.5.2 HDFS Configuration Files . . . 56

4.5.3 CLI Integration with HDFS . . . 58

4.6 MapReduce Strategy: Rationale for `mapPartitions` . . . 59

4.6.1 The Payload Parsing Constraint . . . 59

4.6.2 Candidate APIs and Their Limitations . . . 59

4.6.3 Why was `mapPartitions` chosen? . . . 60

5 Remote Cluster Deployment . . . 62

5.1 Architecture Overview . . . 62

5.1.1 Motivation for a Two-Component Design . . . 62

5.1.2 Repository Structure . . . 63

5.2 The VM Cluster (`cluster/`) . . . 63

5.2.1 Container Configuration . . . 63

5.2.2 Persistent HDFS Storage 64
5.2.3 VM Setup 64
5.3 The Remote Client (`client/`) 65
5.3.1 Overview 65
5.3.2 Pipeline Flow 65
5.4 SSH and SFTP Communication Design 66
5.4.1 The `RemoteCluster` Class 66
5.4.2 Authentication 67
5.4.3 Data Transfer Design 67
5.5 End-to-End Data Flow 67
5.5.1 Separation of Concerns 69
5.5.2 Execution Phase Instrumentation 69
6 Experimental Evaluation 71
6.1 Overview 71
6.2 Experimental Setup 71
6.2.1 Input Dataset 71
6.2.2 Analysis Scenarios 72
6.2.3 Processing Modes: Serial and Parallel 72
6.2.4 Measurement Protocol 73
6.3 Performance Metrics 73
6.4 JVM Warm-up Analysis 75
6.5 Results 76
6.5.1 Quick-Reaction Scenario 76
6.5.2 Launcher Longitude Scenario 76
6.6 Discussion 78
6.6.1 Speedup Analysis 78
6.6.2 Amdahl's Law and Overhead Analysis 79
6.6.3 Throughput Scaling 80
6.6.4 SSH Communication Overhead 80

6.6.5 Threats to Validity 80
6.7 Scalability Analysis 81
6.7.1 Motivation 81
6.7.2 Experimental Setup 81
6.7.3 Theoretical Prediction 82
6.7.4 Results 82
6.7.5 Discussion 82

7 Conclusions 84
7.1 Summary of Contributions 84
7.2 Results Discussion 85
7.3 Limitations 86
7.4 Future Work 87
7.5 Final Remarks 88

1 Introduction

This chapter provides the contextualization, motivation, and objectives for the development of a distributed computing architecture designed specifically for aerospace simulation environments. It begins by examining the evolution of distributed computing from centralized systems to modern cloud-based infrastructures, then identifies the critical challenges in managing large-scale simulation outputs. It also presents the research objective: the design and implementation of AeroSOuDA, a distributed architecture for efficient post-processing and analysis of aerospace simulation results.

1.1 Contextualization

Distributed computing emerged as a vital paradigm to address the fundamental limitations of centralized architecture, particularly when faced with the exponential growth of Big Data, complex simulations, and modern artificial intelligence. For decades, vertical scaling (scale-up) sufficed as hardware manufacturers consistently increased single-core processor speeds in accordance with Moore's Law (Hennessy; Patterson, 2019).

However, physical constraints such as quantum uncertainties and overheating eventually created a severe infrastructure bottleneck, preventing Moore's Law's from staying true (Waldrop, 2016). By shifting the focus to horizontal scaling (scale-out), distributed computing connects an interconnected network of commodity computers, or nodes, to function as a unified system. This approach successfully mitigates the financial burden of high-end hardware, eliminates single points of failure through native fault tolerance, and solves the computational bottleneck by breaking down massive datasets into smaller, parallelizable tasks (Sevilla *et al.*, 2015).

The historical trajectory of this field reflects a steady evolution from isolated mainframes to global, ubiquitous networks. The foundational concepts were established in the 1960s and 1970s with early packet-switching networks like ARPANET, which transitioned in the 1980s into localized clusters tied together by early message-passing interfaces. By the late 1990s, the paradigm expanded into Grid Computing, utilizing idle worldwide resources for grand-challenge scientific problems (Foster; Kesselman, 2003).

The definitive turning point arrived in the 2000s when technology giants introduced frameworks like MapReduce (Dean; Ghemawat, 2004), which later inspired open-source ecosystems such as Apache Hadoop and Spark. Coupled with the rise of modern cloud computing, distributed computing evolved from an exclusive academic tool into the essential backbone of contemporary digital infrastructure, enabling real-time, resilient, and scalable data analytics across the globe.

However, the impact of this MapReduce-driven revolution expanded far beyond commercial data mining and web search algorithms. This divide-and-conquer approach to fragment massive datasets into parallel, independent tasks became the vital engine for accelerating high-fidelity complex simulations. Fields demanding intensive mathematical modeling such as particle physics, computational fluid dynamics and aerospace simulations historically collided with the stark limitations of sequential processing. By translating the divide-and-conquer concepts popularized by MapReduce into High-Performance Computing (HPC) environments (Ekanayake *et al.*, 2010), modern distributed systems can now simulate hyper-complex scenarios in record time (Reed; Dongarra, 2015). What once required months of processing on a single centralized machine is now executed in a matter of hours or minutes across computing clusters, which can also create a bottleneck from post-processing such massive amounts of data.

1.2 Motivation

The amount of simulation data produced nowadays is also growing rapidly, especially in environments where multiple scenarios, parameter sweeps, and repeated executions are needed to support analytical decision making. In many scientific and industrial contexts, the main challenge is no longer only to execute simulations, but also to interpret the large volumes of output that they generate. Once a simulation execution is complete, analysts still need to extract useful information from raw results, compare alternative scenarios, and transform those outputs into metrics that can support robust conclusions. This problem becomes particularly relevant when a simulation batch contains many runs and each run produces detailed records that must later be processed, filtered, and aggregated (Lagares; Rivera; Araya, 2022; Li *et al.*, 2018).

This situation leads to two immediate concerns. The first is how to store large amounts of simulation output in a way that remains scalable and accessible. The second is how to process these outputs efficiently once they have been generated. Even when the analysis required for each individual simulation is conceptually simple, the total cost may become substantial when the number of runs is large. A single machine may become a bottleneck both in storage access and in serial processing time, especially if the analyst must first

move the raw data to a local environment before any meaningful reduction occurs. For this reason, distributed storage and distributed computing appear not as optional refinements, but as natural responses to the scale of the problem (Dean; Ghemawat, 2004; Ghemawat; Gobioff; Leung, 2003).

Because of these challenges, modern computing has produced frameworks and tools that allow analysts as users to express large-scale distributed computations without manually orchestrating every communication and synchronization detail. Different systems emphasize different trade-offs, but most of them aim to provide the same core properties: scalability, fault tolerance, and a programming model that lets the user focus more on the analytical task than on the infrastructure itself. In the context of data-intensive analytics, the MapReduce model provided a foundational abstraction for batch-oriented distributed computation (Dean; Ghemawat, 2004), while later systems such as Spark expanded this model through richer direct acyclic graphs based execution strategies suited to iterative and structured workloads (Armbrust *et al.*, 2015; Zaharia *et al.*, 2012). In this sense, this work is not isolated from the broader history of distributed systems; rather, it is positioned within a long effort to make large-scale data processing both practical and reusable.

For simulation-based decision support, this discussion has a very concrete implication. If the outputs of a simulation batch can be partitioned, stored, and processed in parallel, then the time between executing the simulations and obtaining analytical insight can be reduced substantially. The analyst no longer needs to treat post-processing as a separate, mostly manual phase performed after data retrieval. Instead, the infrastructure itself can participate in the production of metrics and aggregate values. This is precisely the motivation of the present work: to understand how distributed computing ideas can be applied not to the execution of the simulation itself, but to the Analysis and Aggregation (A&A) stage that transforms raw simulation outputs into decision-support products.

1.3 Objective

This work aims to design and implement a distributed computing architecture for aerospace simulation environments. This architecture will be called ”AeroSOuDA”: Aerospace Simulation Output Distributed Architecture. The goal is to integrate the final solution to Aerospace Simulation Ambient (ASA), the project that motivated this work in the first place. In order achieve this goal, some rules must be established:

- The architecture must support a workflow in which the analyst (user) can define or select via interface a metric function and an aggregate function before simulation batch execution;

- The architecture must support any kind of metric and aggregate function;
- The architecture must have a Distributed File System (DFS) to store simulation results;
- The primary language used will be Python, considering the robust ecosystem it brings, allowing us to analyse frameworks such as PySpark;
- The architecture must be tested in a local computer cluster beforehand;
- Considering a simulation batch, simulation results may an come at any order;
- Each metric processing should run preferably in the same node its corresponding simulation was processed in order to reduce data transit via network;
- The aggregate value shall assume at least 2 output types: numbers or graphic plots.

These elements together define the intended contribution of the work: not a new simulation engine, but a new way to transform simulation outputs into useful analytical results more efficiently. By contrast, any improvement toward the simulation processing workflow is outside of the scope. The work does not aim to redesign the simulation engine, change the runtime logic by which individual simulations are executed, or optimize the internal physics or tactical behavior of the simulator.

2 Background

In this chapter, we present the technical background needed to understand the proposed distributed computing architecture and position it within the existing literature.

2.1 The ASA Project

ASA is a software ecosystem created for the modeling and simulation of military operational scenarios to support the development of tactics and procedures in the aerospace context for the Brazilian Air Force (Dantas *et al.*, 2025). It supports scenario modeling and both simulation execution and output analysis via a custom python library named Asapy (Dantas *et al.*, 2024), providing the Brazilian Air Force's higher command with quantitative and statistically robust analyses of outcomes of aerospace defense scenarios. The architectural solution addressed in this thesis concerns a specific bottleneck: while ASA excels at scenario execution, post-processing of large result volumes becomes inefficient when handled serially on a single machine. If the analyst must manually retrieve, structure, and aggregate raw results after each batch, the overall workflow becomes slower and less scalable.

The core contribution of this work is to improve simulation outputs are transformed into analytical products through distributed post-processing, without requiring changes to the core simulation logic.

2.2 Capacity-Based Planning (CBP)

Capabilities-Based Planning (CBP) is a defense-planning approach centered on the capabilities required to accomplish missions under multiple plausible scenarios, rather than on a single predefined threat. In this perspective, the planning process asks which operational effects must be achieved and which combinations of systems, doctrine, personnel, and infrastructure are needed to achieve them robustly across different future situations. As discussed by Davis (2002), CBP is particularly useful when uncertainty is high and

when strategic decisions must remain valid under changing technological, political, and operational conditions.

From an analytical standpoint, CBP depends on methods capable of comparing alternatives under uncertainty. That is why it is commonly associated with techniques such as Delphi-based elicitation (Egfjord; Sund, 2020), and modeling and simulation. In military environments, those techniques are not merely auxiliary tools; they are part of the evidence-generation process through which planners justify capability-development decisions (Hodický *et al.*, 2020).

In an air-defense context, CBP helps transform broad strategic concerns into concrete analytical questions. Instead of asking only which isolated system should be acquired or upgraded, planners may ask which capability mix is required to improve survivability, interception success, warning time, operational flexibility, or resource efficiency. This makes the planning process inherently scenario-based: the same force structure may perform differently depending on radar placement, enemy behavior, rules of engagement, and coordination between assets.

Air-defense planning therefore benefits from analytical environments in which multiple scenarios can be generated, executed, and compared in a structured manner. This is especially important when force design decisions must balance offensive and defensive investments, doctrinal alternatives, and constrained resources. In such cases, CBP works as a bridge between high-level strategic questions and operationally grounded evaluations (Davis, 2002; Hodický *et al.*, 2020).

2.3 Modeling and Simulation (M&S)

M&S refers to the combined use of mathematical and computational representations to abstract real-world systems and replicate their behavior under controlled conditions. A model encodes the essential elements, relationships, and rules that govern a system's dynamics, while a simulation executes that model to generate outcomes over time. M&S has become a fundamental tool across engineering, operations research, and defense, enabling the study of complex systems that are too costly, dangerous, or time-consuming to experiment with directly. By abstracting reality into a computable form, M&S allows analysts to pose hypothetical questions, observe consequences, and reason about system behavior before implementing decisions in the real world.

In the context of operational planning and defense analysis, M&S serves as a bridge between strategic concepts and measurable operational effects. Rather than relying solely on intuition or historical precedent, planners can construct detailed scenarios, execute them repeatedly under different parameter values, and extract empirical evidence about

how systems respond to changing conditions. This evidence generation process is essential for decisions that involve high costs, significant uncertainties, and interdependencies among many variables, precisely the characteristics of capability-based planning in complex domains such as aerospace defense.

Defense-oriented simulation has long been used for analysis, research and development, test and evaluation, and training, precisely because it allows complex systems to be studied before costly real-world decisions are made (Hodson; Hill, 2014). For decision support, the value of M&S lies not only in reproducing an environment, but in enabling structured experimentation. Different strategies, force compositions, parameter ranges, and environmental assumptions can be explored through repeated runs, and the resulting data can be transformed into metrics that support comparison.

2.4 Sensitivity Analysis (SA)

One of the most important kinds of simulation in this context is Sensitivity Analysis (SA). In broad terms, it is a specific kind of simulation that studies how variations in one or more input parameters (called sweep parameters) influence the output of a model. In decision-analysis literature, it is used to evaluate robustness, expose influential parameters, and identify how stable conclusions remain under changing assumptions (Więckowski; Sałabun, 2023). In scenario-based simulation, the same logic applies: the analyst varies a parameter across a batch of runs and observes how the resulting outputs respond, enabling them to evaluate a correlation between two parameters.

For the present work, SA is relevant because it provides a simple and concrete way to understand why simulation outputs need structured post-processing. Each simulation run may generate raw data, but the analyst is usually interested in derived quantities, such as survival time, mission success, or, in the SA case, statistical relationships between a parameter and an outcome. Those derived quantities are not given directly by the simulator; they must be extracted, computed, and aggregated from raw simulation outputs.

To illustrate the context of this thesis, consider the defensive intercept scenario from 2.1, referred here as the Quick-Reaction (QR) scenario:

Two blue fighters must enter enemy territory, reach designated points, and return successfully. Also, three red fighters are activated upon radar detection and engage in combat.

Even in a simple setting such as this one, many analytical questions may arise. For instance, how sensitive is a certain outcome, such as the blue team survival rate, to the radar's longitude? Which variables should be tracked in order to support those compar-

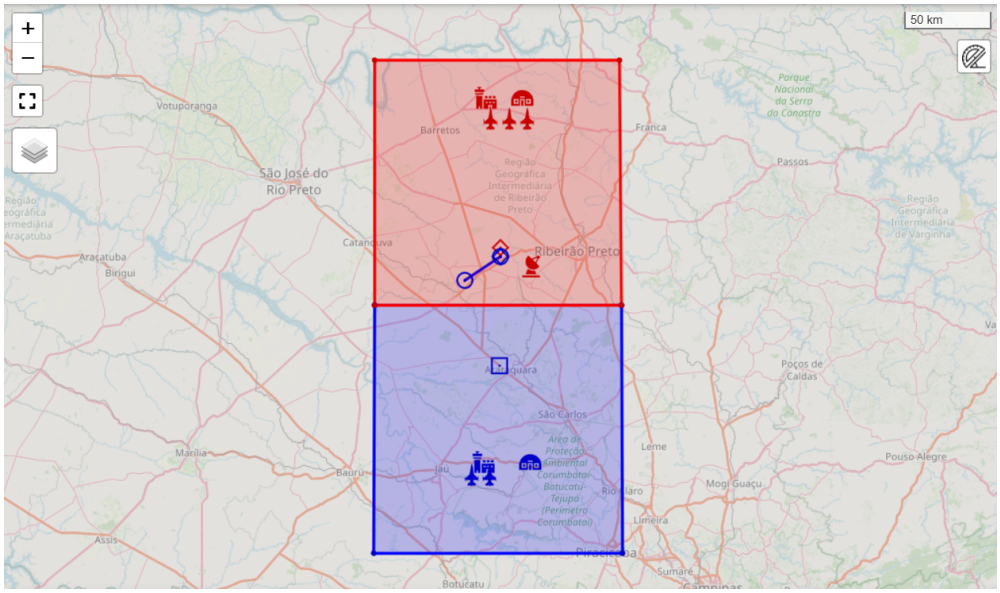


FIGURE 2.1 – Example of a defensive scenario modeled via Webstation.

isons?

This type of scenario is representative of the analytical use case targeted by ASA. The objective is not merely to run one simulation and inspect it visually, but to run a simulation batch, vary selected parameters, and observe how the outcome changes across runs. This is a analytic-wise problem that heavily benefits from M&S.

In the workflow considered in this thesis, it is useful to distinguish three levels of analytical output:

- A result is the raw data produced by an individual simulation run;
- A metric is a useful quantity derived from a single simulation result;
- An aggregate value is obtained by combining metrics from all runs in a simulation batch.

Also, an aggregate function may be defined as the function used to obtain the aggregate value from each metric.

In the QR scenario example, if a simulation batch varying radar longitude is run in order to evaluate its correlation to the survival time of blue jets:

- A result may include, for each frame, whether a fighter remains alive, when radar detection occurred, or when a combat event happened;
- A metric may represent the blue fighter jets' survival time;
- An aggregate value may be the correlation (numerical result) or even a scatterplot between said survival time and the red team radar's longitude (visual result).

2.5 Distributed Computing

Distributed Computing is the field of computer science concerned with the execution of computations across multiple interconnected computing nodes that cooperate to solve a common problem. Instead of concentrating processing, memory, and storage resources in a single machine, distributed computing systems partition workloads among several autonomous nodes that communicate through a network and coordinate their actions through message exchange.

A widely adopted definition characterizes a distributed system as a collection of autonomous computing elements that appears to its users as a single coherent system (Steen; Tanenbaum, 2016). This definition highlights two fundamental properties. First, the participating nodes operate independently, each possessing its own processing capabilities, local memory, and execution environment. Second, despite this physical distribution, the system presents a unified view to users and applications, abstracting the complexity of coordination and communication among nodes.

The main motivations for adopting distributed computing include scalability, fault tolerance, resource sharing, and performance improvement. By distributing data and computation across multiple machines, systems can process larger workloads than would be feasible on a single computer while also reducing the impact of individual node failures. These characteristics have made distributed computing a foundational paradigm for modern large-scale infrastructures, including cloud computing platforms, distributed file systems, big data processing frameworks, and high-performance computing environments.

2.5.1 Distribution Viability in a Simulation Workflow

In simulation-related research, distributed computing appears both during simulation execution and during post-processing stages. Simulation workloads can be partitioned across multiple processors or machines to reduce execution time, while large volumes of generated data can be analyzed using distributed frameworks that exploit parallelism and data locality.

In this work, distributed computing is not used to accelerate the simulator itself, but rather to accelerate the analytical processing of the simulator's outputs. The former corresponds to the Execution Stage, in which simulations are launched and executed. The latter corresponds to the Analysis and Aggregation Stage, in which simulation outputs are processed into metrics and batch-level results.

This distinction is important because the proposed workload is naturally decomposable. A simulation batch consists of many individual runs, and each run generates raw

outputs that can first be processed independently to derive metrics and then combined to obtain a batch-level aggregate value. Such a decomposition is structurally compatible with distributed data-processing strategies.

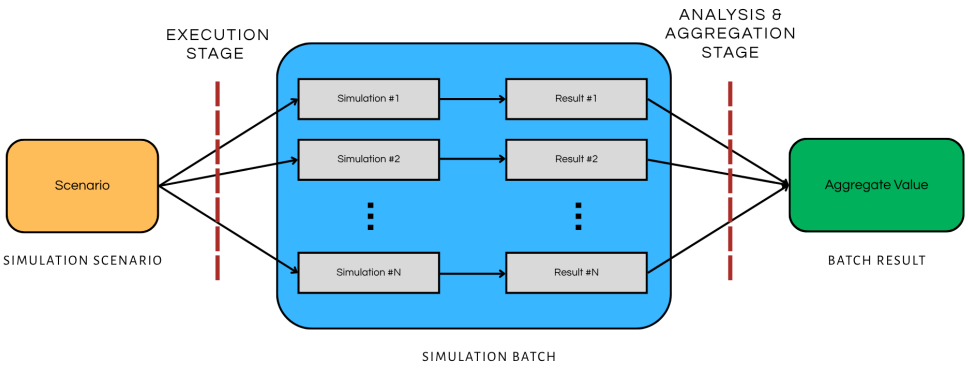


FIGURE 2.2 – General Pipeline for M&S, showing both stages: Execution Stage and Analysis and Aggregation Stage.

In the QR scenario, considering the metrics are implicit inside A&A stage in Figure 2.2:

- The Scenario box represents the QR Scenario;
- Each Simulation box represents a different simulation instance;
- The aggregate value box represents the aggregate value after combining all metrics.

Regarding Figure 2.2 still, although distributed computing can be useful in both stages highlighted in red, this work focuses specifically on the A&A stage.

This choice is justified by the fact that, historically, the scientific efforts of the high-performance computing community have overwhelmingly focused on optimizing the physical execution time and synchronization of simulation engines (Execution Stage) (Fujimoto, 2016). However, with the exponential growth of data volumes generated by large-scale simulations, the true computational and infrastructural bottleneck has shifted to the subsequent post-processing and metric extraction phases (Valduriez *et al.*, 2018; Zheng, 2021). Consequently, while the parallel execution of scenarios has reached a high level of maturity, distributed mechanisms tailored for consolidating batch-level analytical results (Analysis and Aggregation Stage) still lack standardized approaches, presenting a critical gap in the efficiency of modern scientific workflows.

2.5.2 MapReduce

MapReduce is a distributed programming model that decomposes a computation into two main phases: a map phase, in which input records are transformed into intermediate key-value pairs, and a reduce phase, in which values associated with the same key are aggregated (Dean; Ghemawat, 2004). Its importance goes beyond one implementation technology: MapReduce offers a conceptual model for thinking about large-scale batch processing over partitioned datasets.

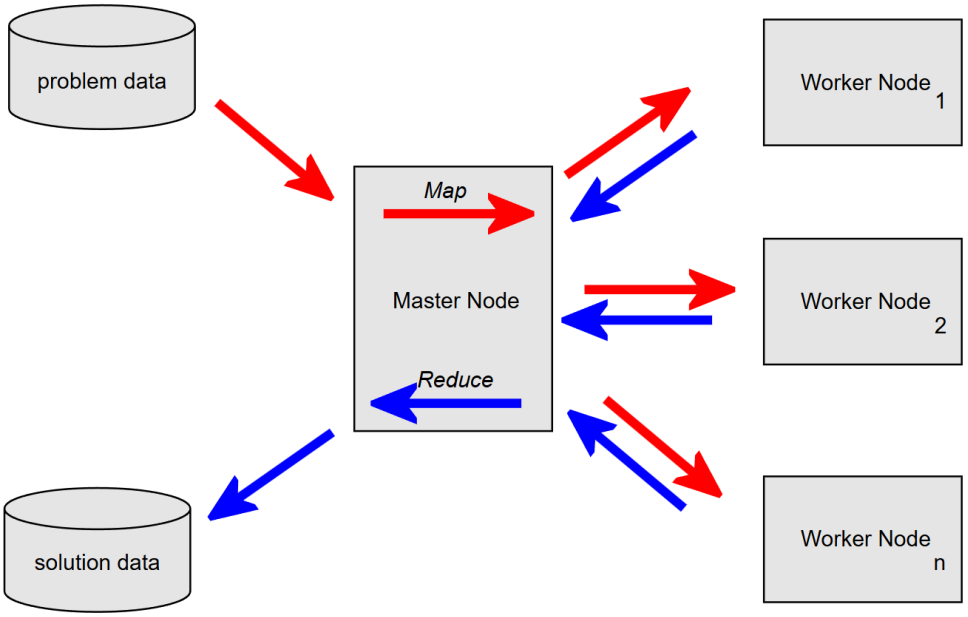


FIGURE 2.3 – MapReduce model overview.

Figure 2.3 illustrates this model. The computation begins with the problem data, representing the input dataset to be processed. A central master node receives the job and is responsible for coordinating the execution. During the map phase, the master partitions the input data into smaller independent tasks and distributes them among multiple worker nodes. Each worker processes its assigned partition in parallel and produces intermediate results. The red arrows in the figure represent the distribution of work from the master to the workers.

Once the mapping tasks have been completed, the intermediate results are returned to the master, as indicated by the blue arrows. During the reduce phase, the master combines, aggregates, or summarizes the partial outputs generated by the workers. Depending on the application, this aggregation may involve operations such as summation, counting, averaging, merging data structures, or computing statistical metrics. The final aggregate value is then written to the solution data, which represents the output dataset produced by the computation.

Although practical MapReduce implementations often include additional stages such

as shuffling, sorting, fault tolerance, and data locality optimization, the fundamental idea remains the same: a master node coordinates a collection of workers that independently process partitions of the input data and subsequently contribute their partial results to a reduction step that generates the final solution. This model enables scalable processing by exploiting parallelism while presenting a relatively simple programming abstraction to the developer.

That conceptual model maps naturally to the problem studied in this thesis. Each simulation output can first be transformed into one or more metrics independently from the others, which resembles the map phase. Then, metrics from all runs in the batch can be combined by an aggregate function, which resembles the reduce phase. Even if the final implementation does not use a canonical Hadoop MapReduce stack, the model remains useful because it provides a clean abstraction for structuring the analytical pipeline.

At the same time, the literature also shows the limits of a purely MapReduce-oriented approach for richer multi-stage analytics. When workflows involve repeated transformations, structured queries, or multiple dependent operations over the same data, models based on richer execution graphs may offer better flexibility and lower overhead (Zaharia *et al.*, 2012). However, these limitations don't pose as a critical problem in our case, since both map and reduce stages can be well translated in our work's analytical pipeline.

2.5.3 Apache Spark

Apache Spark is an open-source distributed computing framework designed for large-scale data processing. Unlike the traditional MapReduce model, which relies on multiple disk-based stages, Spark performs most computations in memory, significantly improving performance for iterative algorithms, interactive analysis, and complex data processing pipelines (Apache Spark Project, 2026; Zaharia *et al.*, 2012).

Spark organizes computation through a cluster architecture composed of a driver process and multiple worker nodes. Applications are expressed as direct acyclic graphs of transformations and actions, allowing the execution engine to optimize task scheduling, data locality, and fault recovery automatically. The framework also provides a unified ecosystem that includes modules for SQL processing, machine learning, graph analytics, and stream processing, making it suitable for a wide range of data-intensive workloads.

A key advantage of Spark in scientific computing environments is its integration with distributed storage systems such as Hadoop Distributed File System (HDFS). By executing computations close to where data are stored, Spark reduces network transfer overhead and enables efficient processing of datasets that exceed the memory and storage capacity of a single machine. These characteristics have motivated its adoption in high-performance

computing and large-scale simulation post-processing workflows (Li *et al.*, 2018).

2.5.3.1 Executors, cores, and partitions

In Spark, an executor is a long-lived worker process responsible for executing tasks and storing intermediate data for a running application. The driver creates tasks from the job plan and sends them to executors, which process data partitions and return partial results. The number of executors defines how many worker processes are available in parallel, while the number of cores per executor defines how many tasks each executor can run concurrently (Apache Spark Project, 2026).

A Spark job is the unit of execution created when an action is triggered, for example `count()`, `collect()`, or a write operation. Internally, a job is split into one or more stages, and each stage is split into tasks that run on data partitions. In practical terms, this means that a single analytical script can launch multiple jobs, and each job consumes the available executor cores according to its stage and partition structure.

Resilient Distributed Dataset (RDD) partitioning controls how the input dataset is divided into independent work units. In practice, each partition is processed by one task, and tasks are scheduled over the available executor cores. As a result, parallel throughput depends on the joint configuration of executors, cores, and partitions. Increasing only one of these parameters does not necessarily improve performance if the others remain constrained.

2.5.3.2 Execution model and the Java Virtual Machine

The execution engine behind Spark is not implemented as a standalone Python runtime. The core of Spark is written in Scala and runs on the JVM, which provides memory management, thread scheduling, object allocation, and garbage collection for the driver and executor processes (Oracle, 2024). This design matters because Spark's in-memory execution model depends on efficient heap usage and low-latency task scheduling. The driver coordinates distributed execution, while the executors are long-lived JVM processes that receive partitions, execute transformations, and return partial results to the driver. In practice, this means that the apparent simplicity of a Python interface hides a more elaborate runtime: PySpark communicates with the JVM-based Spark engine through a local gateway and serializes data between Python objects and JVM data structures (Apache Spark Project, 2026; Oracle, 2024). Without understanding this runtime boundary, many Spark performance and memory issues appear mysterious, even when the user's code is written in Python.

This perspective is particularly relevant for large-scale simulation post-processing. A

batch may contain millions of rows, and a single executor may not be able to hold all intermediate state in memory. Because the executor processes are JVM-backed, the effective capacity of the cluster is determined not only by the number of cores, but also by the configured heap size, the task parallelism, and the data partitioning strategy. In this sense, the JVM is not an implementation detail; it is part of the distributed execution model that determines how work is scheduled and how memory pressure is handled under load (Apache Software Foundation, 2024; Zaharia *et al.*, 2012).

2.5.3.3 Resilient Distributed Datasets

The most important abstraction in Spark is the Resilient Distributed Dataset (RDD) (Zaharia *et al.*, 2012). An RDD is a collection of data partitions that can be processed in parallel across the cluster while retaining a lineage graph that records how the dataset was derived. This lineage is the key to fault tolerance: if a node fails, Spark can rebuild lost partitions by recomputing them from the original input or from a previously derived parent RDD, rather than depending on a central coordinator to persist every intermediate result.

From a conceptual viewpoint, the RDD model is a natural fit for the analytical pipeline in this thesis. The raw simulation output file can be loaded as a distributed dataset, each partition can be transformed independently to extract a metric, and the partial results can be combined through a reduction step. In other words, Spark makes it possible to express the same map-reduce structure introduced earlier in a form that is parallel, fault tolerant, and better suited to repeated transformations over large datasets (Apache Spark Project, 2024; Zaharia *et al.*, 2012).

Even though more recent Spark APIs such as DataFrames and SQL provide higher-level interfaces, the RDD abstraction remains fundamental to understanding the framework. It explains why Spark can support both iterative processing and resilient recovery without requiring every intermediate value to be written to disk, which is one of the central reasons why it is attractive for large-scale analytics and scientific post-processing workflows (Armbrust *et al.*, 2015; Zaharia *et al.*, 2012).

2.5.4 Virtual machines

A virtual machine is a software-defined computer that runs on top of a physical host by emulating the hardware and operating system environment expected by a guest system. In other words, a VM does not require a dedicated physical machine for each service; instead, a single host can allocate isolated execution environments that behave like independent computers for networking, storage, process scheduling, and user access. This

abstraction is especially useful in distributed systems because it allows a team to provision remote computing resources with a predictable configuration while keeping the underlying hardware manageable and reusable (Merkel, 2014; Steen; Tanenbaum, 2016).

From the point of view of the final product of this project, the VM is the component that must be reached remotely. The Spark cluster, the Hadoop filesystem, and the supporting services are not meant to run directly on the analyst's laptop in the production deployment. Instead, the remote VM hosts the distributed processing stack, while the analyst's machine acts as a lightweight client that issues commands and retrieves result artefacts. This architecture makes the VM the actual execution environment for the end-to-end analytical workflow and the node that must be accessed through SSH in the operational system.

This design matters because the VM provides the needed computational resources, persistent storage, and network accessibility that are difficult to guarantee on a personal workstation. It is also the component that normalizes the environment across a project: the same cluster configuration can be deployed on a remote server, monitored remotely, and accessed consistently from different clients without requiring each user to install the full software stack locally.

2.5.5 SSH and SFTP

Secure Shell (SSH) is a network protocol for encrypted remote access to computers and services. It provides authentication, confidentiality, and integrity for command execution over an insecure network, which makes it a natural choice for controlling a remote virtual machine from a local analytical workstation (OpenBSD, 2024a). In the architecture proposed in this thesis, the analyst does not need to run Spark, HDFS, or Docker on their own machine. Instead, the local client connects to the remote Virtual Machine (VM) using SSH, submits commands, inspects the state of the cluster, and triggers execution on the server where the distributed environment is deployed.

Secure File Transfer Protocol (SFTP) is a file-transfer protocol built on top of the SSH transport layer. It allows the client to upload small configuration files, such as scenario definitions in YAML, and to download output artefacts after a job has completed, while preserving the same encrypted communication channel. In practice, SSH is used for remote execution and job orchestration, while SFTP is used for data exchange. This separation is important because file transfer and remote command execution have different operational requirements, but both benefit from the same secure authentication mechanism (OpenBSD, 2024a,b).

This design reduces the complexity of the client side. The analyst interacts with a

lightweight Python client that performs the necessary SSH and SFTP calls to a remote VM, while the computationally expensive data processing remains in the managed distributed environment. This is a standard pattern for cluster administration and remote scientific workflows, and it is especially appropriate when the data volume is large enough that transferring raw simulation outputs to the local workstation would be inefficient or unnecessary.

2.5.6 PySpark

PySpark is the Python interface to Apache Spark, providing a high-level Application Programming Interface (API) for distributed data processing while preserving the analytical flexibility expected in Python-based scientific workflows. In this work's context, its adoption is justified because the analysis and aggregation stage can be naturally expressed as distributed transformations and aggregations over simulation outputs, with better scalability than serial post-processing and without requiring the analyst to abandon the Python ecosystem already used in ASA. In addition, PySpark integrates natively with Hadoop-compatible storage layers, including direct read/write support over HDFS, which is consistent with the data-local architecture proposed for reducing transfer overhead and processing large simulation batches efficiently (Apache Spark Project, 2026; Zaharia *et al.*, 2012).

Finally, related works such as Li *et al.* (2018) reported successful adoption of PySpark combined with HDFS for large-scale simulation data post-processing. The study highlights the suitability of Spark's distributed execution model for performing aggregation and analytical operations over datasets that exceed the capacity of a single workstation. The positive results obtained in such environments provide evidence that the Spark-HDFS ecosystem constitutes a mature and scalable solution for scientific post-processing workloads.

2.5.7 Docker and Containerization

Docker is a platform for operating-system-level virtualization that allows applications and their dependencies to be packaged into lightweight, portable execution environments called containers (Merkel, 2014). At a conceptual level, a container is not a full virtual machine: instead of emulating a complete guest operating system, it shares the host kernel while isolating process trees, filesystem views, network interfaces, and resource limits. This design makes containers substantially lighter than traditional virtual machines, while still providing a degree of isolation that is useful for reproducible software deployment.

The main building block in Docker is the image. An image is a layered, read-only

filesystem template that contains an operating-system base, installed software, configuration files, and the command that should be executed when the container starts. A container is then a runtime instance of that image, with its own isolated namespaces for processes, mounts, and networking. This separation is what allows several services, such as a Spark master, a Hadoop NameNode, and a Python client, to run side by side on the same host without interfering with one another.

From a distributed systems perspective, Docker is especially valuable because it makes the deployment model explicit. Instead of manually installing and configuring each component on a physical machine, the system is expressed as a set of services with defined dependencies, resource constraints, and communication paths. In a local prototype, this means that the distributed stack can be launched as a collection of containers that communicate through a private network and can discover one another by service name. In a more realistic deployment, the same conceptual arrangement can be reproduced on remote hosts, making the cluster logically similar even when the underlying hardware differs.

In practical environments, this multi-service orchestration is commonly implemented with Docker Compose, which allows a complete application stack to be declared in a single YAML file describing services, networks, volumes, startup order, and runtime parameters (Docker, Inc., 2024). Instead of starting each container manually, Compose provides a deterministic way to create and manage the whole topology as one operational unit. For this thesis, that capability is important because the stack includes interdependent services such as Spark, HDFS, and auxiliary scripts. A declarative Compose configuration reduces configuration drift and helps preserve reproducibility between local validation and remote deployment.

The practical relevance of Docker to this thesis is that it supports the reproducibility and portability of the distributed architecture. The local cluster includes multiple services with specific versions of Java, Python, Hadoop, Spark, and supporting tooling; these components must interact consistently for the workflow to be valid. Containerization packages those dependencies in a way that reduces configuration drift and makes the system easier to rerun, inspect, and compare across experiments. In this sense, Docker is not merely a deployment convenience; it is a mechanism that makes the distributed prototype easier to reason about and maintain.

2.6 Distributed File Systems (DFS)

DFS is a storage architecture in which files are partitioned and managed across multiple nodes while being exposed logically as a unified file space. The core motivation of a DFS is to improve scalability, availability, and locality of access.

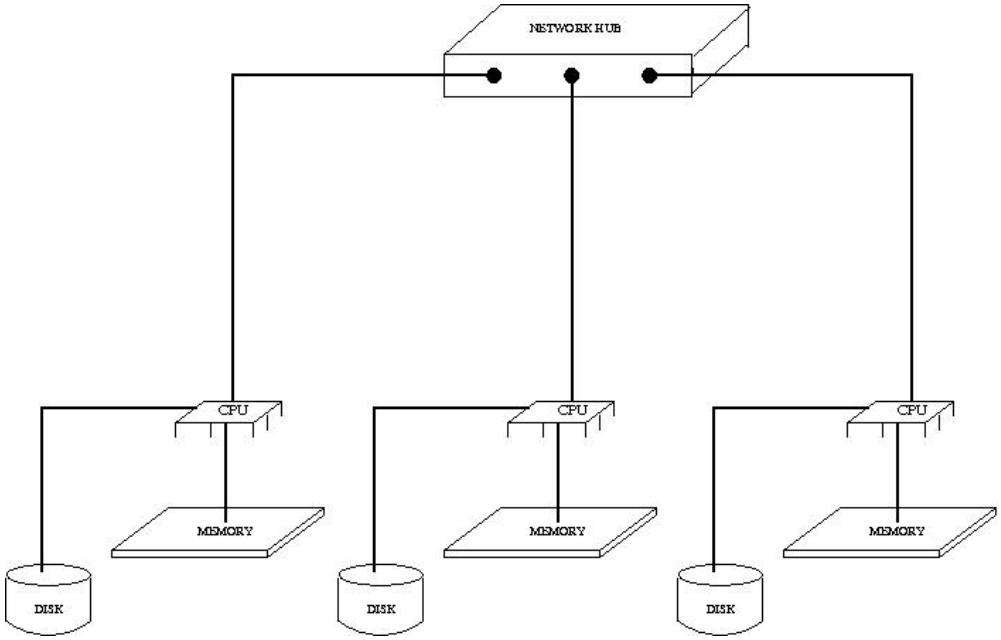


FIGURE 2.4 – Distributed memory overview.

Figure 2.4 presents a simplified distributed-memory system. Unlike shared-memory architectures, where all processors access a common memory space, each node in a distributed-memory system possesses its own local resources, including a CPU, memory, and persistent storage. The nodes are interconnected through a communication network, represented in the figure by the network hub, which enables data exchange among otherwise independent machines. Because each node maintains its own memory and storage, no processor can directly access the memory of another node; instead, information must be transferred explicitly through network communication.

This organization allows the system to scale by adding new nodes, each contributing additional computational power, memory capacity, and storage resources. The aggregate resources of the cluster can therefore exceed the physical limitations of any individual machine. At the same time, the distributed nature of the architecture introduces challenges related to data placement, communication overhead, consistency, and fault tolerance.

Distributed file systems can be viewed as a software layer built on top of such distributed-memory environments. Rather than exposing separate disks attached to individual nodes, a DFS aggregates the storage resources available throughout the cluster and presents them as a unified logical file space.

Consequently, applications interact with files as if they were stored in a single system, while the DFS transparently manages the distribution, replication, and retrieval of data across the underlying nodes.

The Google File System is a foundational example: it was designed to support very large datasets, tolerate commodity-node failures, and keep computation close to stored

data (Ghemawat; Gobioff; Leung, 2003). These same concerns are relevant because simulation outputs can grow into large collections of batch-generated files whose analytical value depends on efficient retrieval and processing.

In the Hadoop ecosystem, HDFS implements this distributed-storage idea in a concrete way. Files are split into blocks and spread across multiple DataNodes, while a dedicated NameNode keeps track of the namespace and block locations. Each block is usually replicated across several nodes, so the failure of one machine does not immediately make the data unavailable. When a Spark job reads a file, the scheduler tries to execute tasks on nodes that already store the required blocks, a design that minimizes network transfer and is commonly referred to as data locality. This is especially important for large simulation outputs, because transferring the complete payload to the analyst's workstation before aggregation would negate much of the benefit of distributed processing.

For the present work, the relevance of a DFS is straightforward. If raw results from many simulations are stored in a distributed file-oriented layer rather than in a single retrieval bottleneck, post-processing can be scheduled close to the output shards themselves. This does not eliminate all communication costs, but it reduces the need to move raw outputs to the analyst before any reduction has occurred. In practical terms, the DFS becomes the persistence layer that supports server-side metric extraction and batch-level aggregation.

Considering similar works such as Li *et al.* (2018) and Morrissey *et al.* (2020), the DFS framework adopted in this work will be HDFS, as it provides high-throughput distributed storage, data locality, built-in fault tolerance, and seamless integration with the Hadoop/Spark ecosystem. In this setting, HDFS is not merely a storage backend: it is the substrate that enables the distributed architecture to treat the batch as a collection of data shards whose processing can be parallelized without central bottlenecks.

2.7 Related Work

The most directly relevant works identified for this thesis are those that address large-scale post-processing of simulation data, distributed scientific analytics, or alternatives to offline post-processing. The comparison developed in this thesis uses criteria that help distinguish both technologies and problem settings:

- Processing paradigm identifies whether the work relies on hybrid HPC and big-data analytics, distributed data platforms, or in-situ / in-transit processing;
- Data access pattern indicates whether the analysis occurs offline after outputs are persisted, during simulation execution, or through distributed query layers;

- Storage and file format help reveal whether the solution expects specialized scientific formats, distributed key-value stores, or more generic file-oriented outputs;
- User interface describes the nature of system access: some systems expose APIs, while others are designed primarily as infrastructure or embedded analysis layers;
- Primary goal specifies the system's objective, whether it is scalable post-processing, distributed visualization, workflow automation, or I/O reduction during execution.

Together, these criteria make it possible to compare the current ASA workflow and the proposed TG architecture against the literature on more than one angle.

Tables 2.1 and 2.2 summarize the nuances between each study to be mentioned in this section.

TABLE 2.1 – Comparison of related works: processing paradigm, data access, and storage (part 1).

Work	Processing Paradigm	Data Access Pattern	Storage / File Format
Li et al. (2018)	Hybrid HPC cluster + Big Data	Simulation output post-processing via HDFS	HPC outputs
Lagares et al. (2022)	Distributed scientific post processing framework	Offline post-processing	CFD output formats
Morrissey et al. (2020)	MapReduce	Post-processing and distributed querying/visualization	HBase/HDFS; large DEM simulation datasets
Friesen et al. (2016)	In-situ / in-transit analysis paradigm	During simulation execution or while data is moving between systems	Simulation data handled inside HPC workflow
ASA Project	Centralized post-processing over single storage; serial metric evaluation	Post-processing of already-generated outputs	Single database instance; raw outputs handled as csv files
This TG proposal	Distributed post-processing architecture	Distributed output post-processing	DFS for outputs; distributed storage of batch results/metrics

Among the selected works, Li *et al.* (2018) and Lagares, Rivera, and Araya (2022) are particularly relevant as they explicitly combine high-performance computing with distributed analytics for post-processing, demonstrating that scalable, distributed, and Python-accessible post-processing is itself a research problem worthy of dedicated system

TABLE 2.2 – Comparison of related works: user interface, primary goal, and simulation domain (part 2).

Work	User Interface	Primary Goal	Simulation Domain
Li et al. (2018)	Jupyter Notebook via JupyterHub	Implement a hybrid workflow that integrates HPC and Hadoop/Spark	Molecular simulation
Lagares et al. (2022)	Python API	Scalable, out-of-core analysis of large simulation results	Turbulent fluid flow / CFD
Morrissey et al. (2020)	Platform/API access layer for queries	Create a platform that enables local processing and results visualization	Discrete Element Method
Friesen et al. (2016)	HPC analysis framework	Reduce I/O and storage overhead	Cosmological / Astrophysics
ASA Project	ASA Webstation + JupyterHub + custom local scripts	Execute simulation batches and allow post-processing of results	Aerospace simulation
This TG proposal	ASA platform + Python API	Parallelize computing of simulation results	Aerospace simulation

design. Morrissey *et al.* (2020) proposes a distributed platform for post-processing and visualization of extremely large DEM datasets, embodying the principle of distributing access, analytics, and visualization over large simulation datasets rather than concentrating all work on the client side. In contrast, Friesen *et al.* (2016) focuses on in-situ and in-transit analysis rather than post-processing over already-persisted outputs, serving as a boundary definition for what this thesis is not attempting. Finally, Deelman *et al.* (2015) demonstrates that complex scientific processing pipelines benefit from explicit workflow representations and fault-aware orchestration across heterogeneous computational resources.

The literature reveals a set of gaps that are highly relevant to the ASA context. First, relatively few works focus on the A&A stage as an autonomous architectural problem. In many cases, attention is concentrated on simulation execution, workflow orchestration in general, or in-situ analysis embedded in the simulator. Second, the air-defense simulation domain itself is much less represented than fields such as molecular dynamics, CFD (Computational Fluid Dynamics), DEM (Discrete Element Method), or cosmology (Friesen *et al.*, 2016; Lagares; Rivera; Araya, 2022; Li *et al.*, 2018; Morrissey *et al.*, 2020).

There are also computational gaps. Many proposed solutions assume tight access to the simulator internals or to specialized scientific formats and infrastructures. Fewer

works address black-box simulators that already produce persisted outputs and still need an analyst-facing post-processing workflow. Likewise, the literature does not strongly emphasize the combination of distributed storage, batch-level metric extraction, user-defined aggregate functions, and Python-centric interaction as a unified architectural problem.

These observations position the present thesis in a specific setting: distributed post-processing of already-persisted simulation outputs in a scenario-based aerospace-defense environment, with a workflow that remains accessible to the analyst.

3 Aerospace Simulation Output Distributed Architecture (AeroSOuDA)

This chapter aims to present the proposed design for AeroSOuDA, the distributed computing architecture for aerospace simulation outputs, comparing the a given current architecture and its struggles with a proposed solution for scalability and performance improvement.

3.1 Current Architecture

The current analytical pipeline considered here is shown in Figure 3.1 and can be summarized as follows:

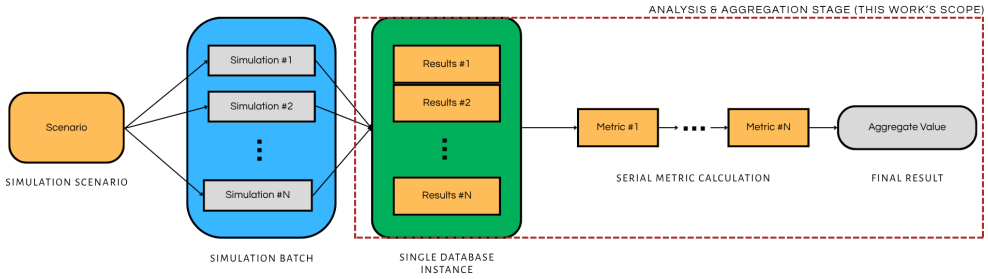


FIGURE 3.1 – Current simulation-processing pipeline.

1. A scenario is created and executed as a simulation batch, producing raw outputs for each individual execution;
2. Those outputs are then stored centrally in a single database instance and must later be processed manually to derive metrics;

3. Metrics are evaluated serially, since information is also retrieved serially from the database;

4. A batch-level aggregate value is manually produced as the final product of the simulation-batch execution pipeline.

In the current workflow, the analyst must retrieve raw outputs, interpret their structure, implement or execute the metric calculation logic, and only then derive the aggregate value of interest. Such a workflow may be manageable for small batches, but it becomes progressively less efficient as the number of runs and the size of the outputs increase.

The problem is not only computational, as it is also architectural. If the data produced by the simulation batch must be moved to the analyst's environment before any meaningful reduction occurs, the system ends up paying a high cost in network data transfer, storage concentration, and manual orchestration. This concept is commonly referred as the moving data paradigm (Partridge, 1992). For an environment whose purpose is to support repeated scenario analysis, this creates friction exactly where analytical agility is most needed.

These limitations motivate a server-side, data-local approach to post-processing. Instead of moving raw simulation outputs to the analyst before any reduction occurs, the processing logic should be moved closer to where the data already resides. This principle is consistent with the broader rationale of the moving-code paradigm (Müller; Bernard; Kadner, 2013).

This thesis thus proposes an architecture implementation in which the simulation processing pipeline satisfies the following characteristics:

- Raw outputs are stored in a distributed manner (via HDFS);
- Metrics are computed in parallel (MapReduce via PySpark);
- Only the analytical products of interest (aggregate values) need to be delivered to the user (via Python API).

3.2 Proposed Architecture

The proposed architecture replaces the current centralized analysis logic with a distributed architecture in which simulation outputs are stored in a DFS and processed in parallel. Conceptually, the workflow remains similar at a high level: simulation results are produced, transformed into metrics, and then combined into a batch-level result. The main difference is where and how these transformations occur. Instead of transferring

large raw outputs to the analyst before reduction occurs, the infrastructure itself should perform the metric extraction and aggregation tasks as close as possible to the stored data.

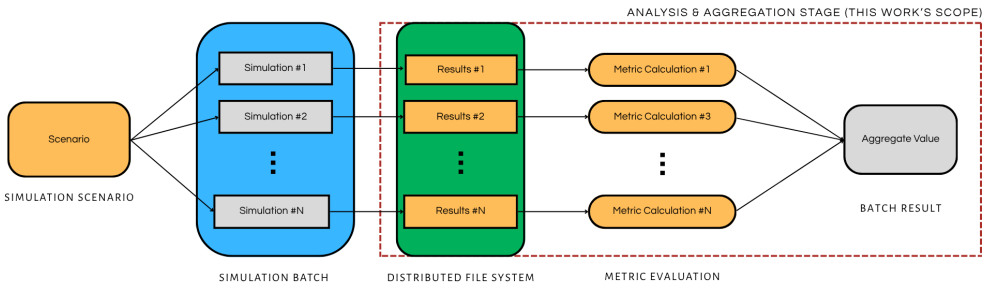


FIGURE 3.2 – Proposed simulation-processing pipeline.

This change is expected to improve the overall workflow in three ways:

1. By reducing unnecessary data movement through network;
2. By allowing independent simulation outputs to be processed concurrently;
3. By allowing the analyst user to request analytical products without manually orchestrating the intermediate steps.

Similar motivations appear in prior work on distributed post-processing and large-scale simulation-data analytics (Lagares; Rivera; Araya, 2022; Li *et al.*, 2018; Morrissey *et al.*, 2020).

4 Local Cluster Implementation

The preceding chapters presented the motivation, conceptual background, and proposed design of Aerospace Simulation Output Distributed Architecture (AeroSOuDA). This chapter describes the first concrete realization of that proposal: a local cluster prototype that validates the architecture's feasibility before deployment on a remote virtual machine. The implementation comprises two cooperating software modules: (1) a CLI application that guides the analyst through the full analytical pipeline; and (2) a distributed processing job, running inside a local Apache Spark cluster, that applies the MapReduce paradigm to extract entity-level metrics and compute batch-level aggregate values. Together, these modules realize the pipeline described in Chapter 3 in a form that can be deployed and tested on a single workstation.

The chapter is organized as follows. Section 4.1 describes the containerized cluster infrastructure. Section 4.2 covers the interactive CLI module. Section 4.3 details the distributed Comma-Separated Values (CSV) processor. Section 4.4 explains the integration between the two modules. Section 4.5 documents the HDFS integration added in a subsequent iteration to replace Docker bind-mount volumes with a proper distributed filesystem. Section 4.6 provides a comparative analysis of Spark API alternatives and justifies the choice of `mapPartitions` for the map phase.

4.1 Cluster Infrastructure

4.1.1 Container Orchestration with Docker Compose

The cluster is instantiated as a set of Docker containers (Merkel, 2014) orchestrated by Docker Compose, a tool that defines and runs multi-container Docker applications from a single declarative configuration file. Docker enables each node to run as a lightweight, isolated process that shares the host operating-system kernel but maintains a separate filesystem, network interface, and resource allocation. This containerisation approach reproduces the logical properties of a distributed cluster without requiring dedicated physical hardware, making the prototype easy to deploy and reproduce on any workstation

with Docker installed.

The current cluster configuration defines thirteen containers:

- spark-master: the Spark standalone master node, responsible for accepting application submissions, scheduling tasks, and assigning work to the registered worker nodes. It exposes the Spark web User Interface (UI) on port 8080.
- spark-worker-1 through spark-worker-6: six Spark worker nodes, each running one executor process. Workers register with the master on startup and report their available processing cores and memory. All six workers share a common container image and differ only in their hostnames.
- spark-history-server: Spark server used to reconstruct completed job timelines.
- namenode: the HDFS NameNode, which maintains the filesystem namespace and block-location metadata.
- hdfs-datanode-1 through hdfs-datanode-4: four HDFS DataNode containers, each storing raw data blocks on local storage and serving read and write requests directly to Spark executors. The replication factor is set to two, so each block has one primary copy and one replica distributed across the four DataNodes.

All containers are connected through a Docker bridge network, enabling inter-container communication using container names as resolvable hostnames. Volume mounts expose the host's `inputs/`, `outputs/`, and `scenarios/` directories inside the `spark-master` container, so the Spark job can read input data and write results without explicit file transfers under normal operation.

4.1.2 Scenario Configuration Format

A key design decision in this implementation is the use of YAML for scenario configuration files, in contrast to the JavaScript Object Notation (JSON) format used by the ASA ecosystem for simulation scenario definitions. While JSON is a strict, minimal data-interchange format well suited for machine-generated content, YAML is a human-readable data-serialisation language that supports inline comments, multi-line strings, and a more forgiving syntax (Ben-Kiki; Evans; Ingerson, 2009). Because scenario configuration files are intended to be created, inspected, and occasionally modified by analysts, YAML reduces the authoring cost significantly.

The raw simulation scenario inputs exported from the ASA Webstation environment are in JSON format. These files are processed by the ASA toolchain to produce the CSV result files consumed by the distributed processor presented in this thesis.

4.1.2.1 Annotated YAML Reference

Each scenario configuration file is a plain-text YAML document. An example using the Quick-Reaction scenario is shown below; Section 4.1.2.2 explains every field:

```
name: "Quick-Reaction"
description: >
  Pearson correlation between red radar longitude
  and blue fighter survival time.

x_variable:
  name: "red_radar_longitude"
  entity_selector:
    side: "red"
    type_contains: "radar"
  field_path: "state.geo_pos.longitude"
  frame_aggregation: "last"
  scope: "per_experiment"

y_variable:
  name: "blue_fighter_survival_time_frames"
  entity_selector:
    side: "blue"
    type_contains: "fighter"
  field_path: "frame"
  frame_aggregation: "max"
  scope: "per_entity"

batch_aggregation:
  type: "pearson_correlation"
```

4.1.2.2 Field-by-Field Reference

Table 4.1 provides a complete reference for all supported fields within an `x_variable` or `y_variable` block. The `batch_aggregation` block is described separately in Section 4.1.2.4.

It is important to mention beforehand the program outputs generated by this project. Each execution produces 3 files: a JSON summary file containing execution metadata and aggregate values, an aggregated CSV table with the processed analytical dataset, and a

TABLE 4.1 – Scenario YAML field reference for `x_variable` and `y_variable`.

Field	Required	Type	Description and options
`name`	Yes	string	Identifier used in the output CSV column header and in the scatter plot axis label. Used exclusively for describing the variable.
`entity_selector.side`	Yes	string	Restricts extraction to entities on the specified team. Must be one of `"blue"` or `"red"`.
`entity_selector.type_contains`	No	string	Case-insensitive substring that must appear in the entity type field (e.g. `"fighter"`, `"radar"`, `"F-16C"`). If omitted, all entities on the specified side are matched.
`entity_selector.name_contains`	No	string	Case-insensitive substring that must appear in the entity name field. Used when multiple entity types share the same side and type but differ only by name.
`field_path`	Yes	string	Dot-notation path to the numeric value within the `payload` JSON column of the CSV. The special value `"frame"` returns the integer frame number directly (useful for computing survival time via `max` aggregation). See Section 4.1.2.3 for how to discover valid paths.
`frame_aggregation`	Yes	enum	Strategy for combining multiple row values for the same entity within one experiment into a single observation. Options: `"max"` (highest value across all frames), `"min"` (lowest value), `"last"` (value at the highest frame number), `"first"` (value at the lowest frame number).
`scope`	Yes	enum	Granularity of the resulting data point. `"per_entity"`: one observation per (entity, experiment) pair, suitable for entity-level analyses when multiple entities (e.g. two fighters) are tracked in each run. `"per_experiment"`: one observation per experiment, suitable for when the independent variable is uniform within an experiment (e.g. the radar's longitude is the same for all entities in the same run).

PNG image with the scatter plot used for visual inspection of the x-y relationship. These three files are the expected outputs referenced throughout this chapter.

4.1.2.3 Discovering the `field_path`

The `field_path` value must correspond to a valid key path inside the `payload` column of the simulation CSV. Since the payload is a nested Python-literal or JSON dictionary, the structure depends on the entity type and must be inspected empirically. The recommended workflow for discovering valid paths is:

1. Open the output CSV from a completed batch execution in a Python environment.
2. Filter rows to the entity type of interest;
3. Inspect the resulting dictionaries to identify the desired field and its path;
4. The dot-notation path is the concatenation of dictionary keys from root to leaf: `state.geo_pos.longitude`, `state.damage`, `mach`, etc.

It is important to note that some simulation parameters set as sweep parameters in the JSON file may not appear directly as entity state fields in the payload column of the CSV. Every row of the output CSV carries the entity's runtime state at a given frame, but it does not carry the experiment-level parameter values that the analyst configured in the batch before execution. This is a limitation of the current YAML scenario schema: any `field_path` must correspond to a path that exists inside an individual payload dictionary. This is further discussed at Section 7.3.

4.1.2.4 Batch Aggregation and Pearson Correlation

The `batch_aggregation` block specifies how the per-entity or per-experiment (x_i, y_i) pairs extracted across all experiments are combined into a single batch-level result. The `type` field accepts one of four string values:

- `"pearson_correlation"`: computes the Pearson product-moment correlation coefficient r between the x and y columns (see Section 4.1.2.5). This is the primary analytical output of this work and the type with full statistical interpretation.
- `"scatter_only"`: generates the scatter plot without computing any numeric aggregate. Useful for visual exploratory analysis before choosing a statistical method.

- `"mean_y"`: computes the arithmetic mean of all y observations across the batch. Useful when the goal is to characterise average entity behaviour rather than a relationship between two variables.
- `"count_y"`: counts the number of valid y observations. Useful as a diagnostic to verify that the entity selector is matching the expected number of entities.

Within the scope of this work, Pearson correlation is the only batch aggregation type used in the experimental evaluation. The other types (`scatter_only`, `mean_y`, `count_y`) are implemented in the codebase (`processor/output.py`) and can be activated by changing the `type` field in the YAML without any modification to the processing infrastructure. Extending the system to support richer aggregation functions such as Spearman rank correlation, polynomial regression, or multi-variable analysis, would require adding the corresponding computation to the `compute_batch_aggregate` function in `processor/output.py` and declaring the new type name in the scenario configuration schema.

4.1.2.5 Mathematical Foundation: Pearson Correlation Coefficient

The Pearson product-moment correlation coefficient, introduced by Karl Pearson in 1895 (Pearson, 1895), measures the degree of linear association between two continuous random variables X and Y. For a sample of n paired observations $(x_1, y_1), \ldots, (x_n, y_n)$, it is defined as (Montgomery; Runger, 2014):

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{4.1}$$

where $\bar{x} = \frac{1}{n}\sum x_i$ and $\bar{y} = \frac{1}{n}\sum y_i$ are the sample means. An equivalent formulation expresses r in terms of sample covariance and standard deviations:

$$r = \frac{\hat{\mathrm{Cov}}(X, Y)}{\hat{\sigma}_X \cdot \hat{\sigma}_Y} \tag{4.2}$$

The coefficient satisfies $r \in [-1, +1]$ for all finite samples. Its interpretation is:

- $r = +1$: perfect positive linear relationship, y increases proportionally with x.
- $r = -1$: perfect negative linear relationship, y decreases proportionally as x increases.

- $r = 0$: no linear association (the variables may still be related in a non-linear manner).
- $|r| > 0.7$ is commonly interpreted as a strong linear association; $0.3 < |r| \leq 0.7$ as moderate; $|r| \leq 0.3$ as weak (Montgomery; Runger, 2014).

Comparison with other correlation measures. Pearson's r captures only linear relationships and is sensitive to outliers because it uses raw values. Two alternative rank-based measures are commonly used when linearity cannot be assumed or the data contains extreme values:

- Spearman's ρ replaces raw values with their ranks before applying Equation (4.1). It captures any monotonic relationship (not only linear) and is robust to outliers. It is the appropriate choice when the relationship is expected to be non-linear but consistently increasing or decreasing.
- Kendall's τ is based on the number of concordant and discordant observation pairs. It is also non-parametric and tends to be more interpretable when sample sizes are small.

Pearson's r was chosen for this work because the analytical scenarios are designed as sensitivity analyses in which the independent variable (e.g. radar longitude) is swept over a continuous range and the response (e.g. fighter survival time) is expected to vary monotonically and approximately linearly with the input. The choice of Pearson r is consistent with similar distributed simulation post-processing works (Lagares; Rivera; Araya, 2022; Li *et al.*, 2018), in which scalar correlation between a swept parameter and a performance metric is the primary analytical product.

4.1.2.6 Creating a Scenario YAML from Scratch

To create a new scenario configuration from scratch, the analyst should follow these steps:

1. Identify the analytical question. The YAML encodes one bivariate question of the form "how does [x entity state] relate to [y entity state] across experiments?". Example: "does the radar's longitude correlate with damage taken by the opposing team fighters?"
2. Identify the x entity. Consult the Scenario JSON file ,which describes all components and their sides and types, to find the entity whose attribute was swept as the alias. The entity type string used in `type_contains` should be a substring of the `type` field found in the payload `id` sub-dict.

3. Discover the `field_path` values. Follow the procedure described in Section 4.1.2.3: load a sample of the output CSV in Python, filter by `asa_type = 2` and the target entity type, and inspect the parsed payload dictionaries to identify the desired key paths.

4. Choose `frame_aggregation` and `scope`.

   For entity attributes that are constant throughout the simulation (e.g. radar position), choose `frame_aggregation: "last"` and `scope: "per_experiment"`. For evolving attributes (e.g. fighter damage that accumulates over time), choose `frame_aggregation: "max"` (or `"last"`) and `scope: "per_entity"`.

5. Save the file with a `.yaml` extension in the `scenarios/` directory. When running the CLI, select the file from the interactive menu or pass it via the `-scenario-config` argument. The wizard mode (option 7a in the pipeline) can also generate a valid YAML from guided prompts, which the analyst can then inspect and manually adjust.

4.2 The Command-Line Interface Module

4.2.1 Overview

The CLI module (`cli.py`) is the primary user-facing entry point for the analyst. It orchestrates the full analytical pipeline through a sequence of interactive prompts, removing the need to manually manage Docker containers, craft Spark submission commands, or handle file transfers. The module uses the Rich library (McGugan, 2024) for styled terminal output including coloured panels, tables, and live progress indicators and the questionary library (Technologies; Contributors, 2024) for interactive selection menus and text prompts.

The module is designed to run entirely in the terminal, requiring no graphical interface. Each stage of the pipeline is presented as a guided step, with descriptive messages, validation feedback, and error panels that explain the cause of any failure and suggest corrective actions.

4.2.2 Pipeline Flow

Figure 4.1 presents the CLI pipeline as a flowchart. The pipeline is composed of nine sequential stages, one of which contains a decision branch routing to different sub-flows depending on analyst choices.

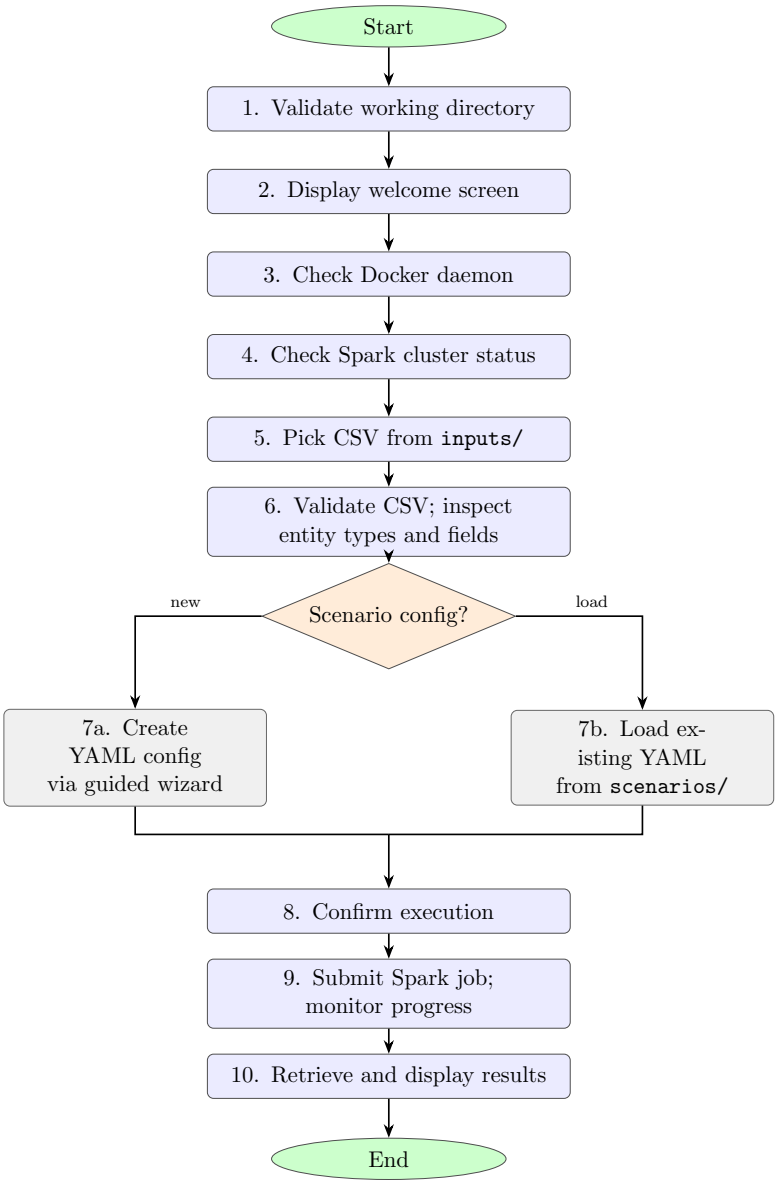


FIGURE 4.1 – Execution pipeline of the CLI module (`cli.py`).

The pipeline's stages are described as follows:

1. Working directory validation. Before any user interaction, the module verifies that it is being executed from the `localcluster/` directory, where `docker-compose.yml` and all required subdirectories reside. If the script is launched from an incorrect location (for example, from an external disk that Docker cannot mount), the pipeline exits immediately with a descriptive error message.

2. Welcome screen. A styled panel introduces the tool and lists the pipeline steps.

3. Docker daemon check. The module verifies that the Docker daemon is running and accessible by issuing a test `docker ps` command. If the daemon is not running, the program aborts, requiring the user to start it beforehand.

4. Cluster status check. The CLI verifies that the Spark cluster is running and reachable by querying the Docker daemon for the status of the `spark-master` container. If the cluster is not running, the pipeline exits immediately with a clear panel that instructs the analyst to start the cluster manually via `docker compose up -d` before re-invoking the tool. This policy was adopted to keep the CLI within the minimal viable scope: it avoids requiring CLI users to have elevated permissions for automatic daemon manipulation and eliminates a class of silent failures that can arise when the cluster master restarts independently of the workers. The analyst is responsible for starting the cluster once before running the pipeline; the CLI verifies its availability and fails fast if it is absent.

5. CSV selection. The analyst selects a pre-existing CSV file from the `inputs/` directory. The module lists all `.csv` files found in that directory together with their sizes and lets the analyst choose one interactively. If the directory is empty, the pipeline exits with guidance on how to place a simulation result file there.

6. CSV validation and entity inspection. The selected CSV is sampled (up to 10 000 rows) and validated against the required column schema. The schema requires twelve columns: `id`, `frame`, `execution_id`, `mixr_type`, `asa_type`, `asa_custom_type`, `payload`, `sim_time`, `utc_time`, `execution_uuid`, `batch_id`, and `experiment`. Missing columns are listed and the pipeline exits with a descriptive error. Valid files proceed to entity inspection: the `payload` column of each sampled row is parsed to discover entity types, and a table is displayed showing each entity's side, type name, representative names, and available nested field paths.

7. Scenario configuration. The analyst selects or creates a YAML scenario configuration file. Two options are offered: loading an existing file from the `scenarios/` directory (with a human-readable preview of its content), or creating a new one

through a guided wizard. The wizard asks a sequence of plain-language questions first the Y variable (the metric of interest), then the X variable (the independent parameter), and finally the batch aggregation function and internally translates the answers into a YAML file saved to `scenarios/` for future reuse. The analyst is never required to understand or manually edit YAML syntax.

8. Execution confirmation. A summary panel lists the selected CSV file, the scenario name, the X and Y variable definitions, and the output directory. The analyst explicitly confirms before the job is submitted.

9. Spark job submission and monitoring. The job is submitted by executing the `run_csv_processor.sh` wrapper script inside the `spark-master` container via `docker compose exec`. The standard output and error streams of the container process are captured simultaneously by two reader threads. Spark's stage-progress lines formatted as `[Stage N: ... (done + active) / total]` are matched by a regular expression, and the derived completion percentage is displayed as a live progress indicator with an elapsed-time counter. Notable status messages printed by the processing script are also surfaced to the progress display.

10. Result retrieval and presentation. After the job terminates, output files are copied from the container's temporary directory to the host `outputs/` directory via `docker cp`. A final panel reports the scenario name, the number of data points produced, the batch aggregate value (for example, the Pearson correlation coefficient), and the paths to the generated CSV, PNG, and JSON summary files. If the job terminates with an error, a failure panel instead shows the last log lines from the container process.

4.3 The Distributed CSV Processor

4.3.1 Overview

The CSV processor (`scripts/csv_processor.py`) is the core distributed computation module. It is submitted to the Spark cluster as a PySpark application (Apache Spark Project, 2026) via the `run_csv_processor.sh` wrapper script and executes entirely within the Docker cluster. The processor reads the input CSV, applies the MapReduce paradigm to extract per-entity metrics and per-experiment x-variable values, computes the batch-level aggregate, and writes three output artefacts: an aggregated CSV, a scatter plot image, and a JSON summary.

4.3.2 Scenario Configuration Resolution

Before any data is read, the processor resolves the scenario configuration by following a three-level priority order:

1. The `-scenario-config` command-line argument, if provided and the file exists;
2. The default container path `/opt/spark/scenarios/`, if the folder exists;

In normal operation, the CLI module injects the analyst-selected YAML content directly into the container via a Python one-liner piped through `docker compose exec`, bypassing potential bind-mount overlay limitations that can prevent the volume-mounted `scenarios/` directory from being writable inside the container.

4.3.3 Spark Ingestion and Validation

A `SparkSession` is created with the application name derived from the scenario name. The input CSV is loaded using Spark's native CSV reader with schema inference enabled, configured for single-line payload parsing and permissive mode (malformed rows are silently skipped rather than causing the job to abort). Immediately after loading, the column set is validated against the required schema; missing columns cause the job to abort with a descriptive error message.

After validation, a base DataFrame is constructed by selecting only the three columns required for processing: `experiment` (cast to long integer), `frame` (cast to double), and `payload` (cast to string). Rows with null experiment values are filtered out, and the DataFrame is repartitioned to the configured partition count to distribute work across the four available Spark workers.

4.3.4 Distributed Map Phase

The map phase is implemented using Spark's `mapPartitions` transformation, which applies a Python function to each RDD partition independently on the worker node that holds that partition (Zaharia *et al.*, 2012). This design amortises per-row Python call overhead across the entire partition, rather than invoking a separate function for each individual row.

The partition function is constructed by a factory that captures the x and y variable configurations in a closure, which is serialised and broadcast to each Spark executor at job dispatch time. Within each partition the function proceeds as follows:

1. It parses the `payload` string for each row, supporting both Python-literal-style dictionaries (`ast.literal_eval`) and strict JSON.

2. It extracts the entity identifier, side (blue or red), name, and type from the parsed payload.

3. It checks whether the entity matches the selector defined in the x or y variable configuration by side, type substring, and optional name substring.

4. If matched, it extracts the target field value by traversing the nested payload dictionary using a dot-notation path. The special path `"frame"` returns the row's frame number directly, enabling survival-time computation via `max` aggregation.

5. It updates a partition-local dictionary with the extracted value, applying the configured frame-aggregation strategy (`max`, `min`, `last`, or `first`) across successive rows sharing the same entity and experiment key.

At the end of each partition the function yields two items: the partial x-variable dictionary and the partial y-variable dictionary.

4.3.5 Reduce Phase and Output Generation

The partial dictionaries from all partitions are collected on the Spark driver via a single `collect()` call. The driver iterates over all collected items and merges them using the same aggregation logic applied within partitions, producing two global dictionaries: one for x and one for y. The two dictionaries are then joined into a tidy output DataFrame by matching experiment identifiers, and also entity keys when the observation scope is `per_entity`. Experiments for which either variable could not be extracted are excluded from the output.

The batch-level aggregate is computed from the output DataFrame according to the type specified in the scenario configuration:

- Pearson correlation: yields a coefficient $r \in [-1, +1]$ between the x and y columns, appended as an additional column in the output CSV.
- Mean of y: computes the arithmetic mean of all y observations.
- Count of y: counts the total number of y observations.
- Scatter-only: produces no numeric aggregate; only the scatter plot is generated.

Three output artefacts are written to a temporary directory inside the container:

- An aggregated CSV containing one row per (experiment, entity) pair with the extracted x and y values and, for Pearson correlation, the batch-level coefficient appended as an additional column.
- A scatter plot image (`.png`) displaying the x variable on the horizontal axis and the y variable on the vertical axis, with one point per data row.
- A JSON summary file containing the scenario name, input path, output paths, number of data points, and the aggregate value with its type and any reason for a null result.

After the Spark job finishes, the CLI module copies these artefacts from the container's temporary directory to the host `outputs/` directory via `docker cp`, making them accessible outside the Docker environment.

Figure 4.2 summarises the distributed processing pipeline.

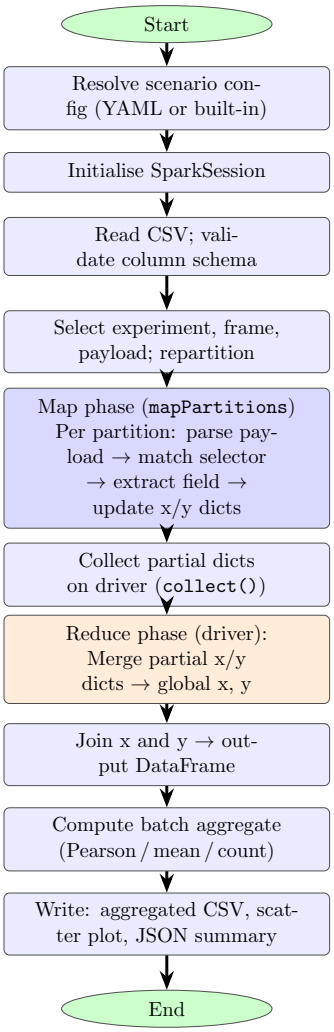


FIGURE 4.2 – Distributed processing pipeline of the CSV processor (`scripts/csv_processor.py`). The map phase runs in parallel across the six Spark worker containers; the reduce phase executes on the Spark master.

4.4 Module Integration

Figure 4.3 presents the data-flow relationships between the two modules and the external systems they interact with. The CLI module is the sole point of contact between the analyst and the system; it drives the execution of the processing module as container commands and manages all data movement between the host and the Docker cluster.

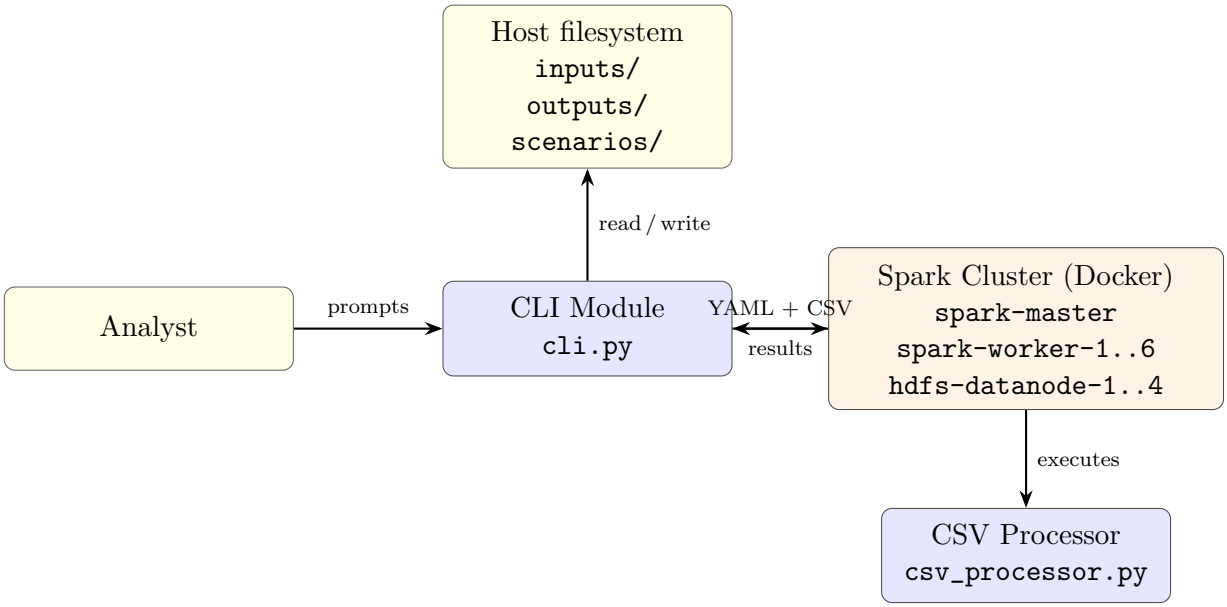


FIGURE 4.3 – Data-flow integration diagram of the AeroSOuDA local cluster modules.

The analyst places pre-existing CSV simulation result files in the `inputs/` directory on the host filesystem. The CLI injects the analyst-selected YAML scenario configuration directly into the `spark-master` container and uploads the input CSV to HDFS, then submits the processing job by executing `run_csv_processor.sh` inside the container.

The job runs on the local Spark cluster: the six worker containers process partitions of the input CSV in parallel during the map phase, yielding partial x and y aggregation dictionaries to the master, which executes the reduce phase and writes the output artefacts to a temporary directory. The CLI then copies the artefacts from the container to the host `outputs/` directory via `docker cp` and presents the results to the analyst.

This end-to-end flow implements the three-stage pipeline proposed in Chapter 3 distributed storage (via HDFS), parallel metric extraction (MapReduce via PySpark), and analyst-facing result delivery (via the CLI) in a self-contained, locally deployable form that validates the proposed AeroSOuDA architecture before its deployment on a remote virtual machine, as described in Chapter 5.

4.5 HDFS Integration

4.5.1 The Storage–Compute Separation Principle

A fundamental design question in distributed computing is how to separate the storage layer where data resides from the compute layer where data is processed. In the AeroSOuDA prototype, the compute layer is provided by Apache Spark and the storage layer by HDFS.

The cluster is configured with four DataNode containers and six Spark worker containers, providing block replication across DataNodes with a replication factor of two and distributing compute work across six parallel executors. This configuration was used for both the local proof-of-concept tests and the remote cluster evaluation described in Chapter 5 and Chapter 6.

4.5.1.1 Full Interaction Protocol

When the CLI submits a job, the following sequence of interactions takes place between the cluster entities:

1. Job submission. The CLI runs `docker compose exec spark-master spark-submit ...`, which spawns the driver process on the master container.

2. HDFS metadata query. The driver opens a Hadoop `FileSystem` client and sends an Remote Procedure Call (RPC) to the NameNode on port 9000: "what blocks does `results.csv` have, and on which DataNode are they stored?"

3. NameNode response. The NameNode consults its in-memory namespace and replies with the block identifiers and the DataNode address. This is pure metadata; no data bytes pass through the NameNode.

4. Resource allocation. The driver requests executor slots from the master. The master checks its worker registry and confirms which slots are free, then instructs the workers to launch executor processes.

5. Task dispatch. The master sends `LaunchTask` RPC messages to the workers, assigning each a set of partitions to process.

6. Block fetch. Each worker's Hadoop client opens a direct TCP connection to the DataNode and streams the raw bytes of its assigned partition. No data passes through the master or the NameNode.

7. MapPartitions execution. The worker executes the `_aggregate_partition` function on its partition iterator entirely locally, with no network communication and builds a partition-local `(x_local, y_local)` accumulator dictionary.

8. Result collection. The `collect()` call transfers all partial dictionaries from the workers to the driver over the Spark shuffle channel. Because each dictionary has at most one entry per experiment, the transferred data volume is $O(E)$ where E is the number of experiments, not $O(N)$ where N is the total number of rows.

9. Reduce phase. The driver merges all partial dictionaries, builds the output table, computes the batch aggregate, and writes the three output artefacts to a temporary directory inside the master container.

10. Completion reporting. Workers acknowledge task completion to the master, which updates its slot availability.

11. Output retrieval. The CLI issues `docker cp` to copy the output files from the master container to the host `outputs/` directory.

Two background protocols run concurrently throughout a job: Spark worker heartbeats to the master (every ten seconds, confirming executor liveness) and DataNode heartbeats to the NameNode (every three seconds, confirming block availability). Failure to deliver three consecutive heartbeats causes the master to mark the worker as lost and reschedule its pending tasks.

The interplay between the two systems is further mediated by the Hadoop client libraries bundled with the `apache/spark` image. In a multi-node cluster, Spark's data locality scheduler can prefer tasks on workers that co-reside with a DataNode holding the required blocks, reducing network traffic. In the local Docker prototype all containers share the same host machine, so locality is always "RACK_LOCAL" (same rack, different process) at best, but the architecture correctly prepares for a future multi-node deployment.

4.5.2 HDFS Configuration Files

The `conf/` directory contains four files that configure both HDFS and Spark layers:

`conf/hadoop.env`. An environment-variable file loaded by the `bde2020/hadoop-namenode` and `bde2020/hadoop-datanode` images via `env_file` in the Docker Compose definition. Key entries:

- `CORE_CONF_fs_defaultFS=hdfs://namenode:9000`, sets the default filesystem URI for Hadoop processes running inside the container.

- `HDFS_CONF_dfs_replication=2`, sets the block replication factor; two is appropriate for a four-DataNode cluster, providing one redundant copy per block while remaining below the number of available DataNodes.
- `HDFS_CONF_dfs_permissions_enabled=false`, disables HDFS user-permission enforcement, simplifying the local development workflow.
- `HDFS_CONF_dfs_blocksize=134217728`, sets the block size to 128×2^{20} bytes. The block size is the unit of storage, not a file-size limit. A 66 MB CSV occupies one block; a 500 MB file is split into four blocks distributed across DataNode containers. The only practical file-size constraint is the total DataNode disk capacity.

`conf/core-site.xml`. The standard Apache Hadoop core configuration file, understood by all Hadoop components and clients. It is bind-mounted inside every Spark container at `/etc/hadoop/core-site.xml`, and the environment variable `HADOOP_CONF_DIR=/etc/hadoop` is set in each Spark service. This causes the Hadoop client libraries embedded in the Spark distribution to load this file at startup, making the `hdfs://namenode:9000` URI resolvable from any Spark executor. The file also sets `dfs.client.use.datanode.hostname=true`, instructing the Hadoop client to resolve DataNode addresses by container hostname rather than by IP address, which is required for correct routing inside Docker bridge networks.

`conf/hdfs-site.xml`. The standard Hadoop HDFS configuration file, bind-mounted into the HDFS containers. It reiterates the block size, replication factor, and DataNode hostname settings in the XML format expected by the NameNode and DataNode processes.

`conf/spark-defaults.conf`. The Spark-wide configuration file, bind-mounted into all Spark containers at `/opt/spark/conf/spark-defaults.conf`. Beyond the partition and memory tuning parameters, two entries support the HDFS integration:

- `spark.executorEnv.PYTHONPATH=/opt/spark/scripts`, adds the volume-mounted scripts directory to the Python path of every executor worker, making the `processor/` package importable without `-py-files` packaging.
- `spark.eventLog.dir=file:///opt/spark/events`, directs event logs to the bind-mounted `events/` directory on the host, which the Spark History Server uses to reconstruct completed job timelines.

A deliberate design decision is that `spark.hadoop.fs.defaultFS` is not set globally in `spark-defaults.conf`. If it were, every Spark file reference including the Python script path in `spark-submit` would be resolved against HDFS, breaking local-path references. Instead, the `run_csv_processor.sh` wrapper script injects the flag `spark.hadoop`

`.fs.defaultFS=hdfs://namenode:9000` only when the input path begins with `hdfs://`. For local input paths it instead injects `spark.hadoop.fs.defaultFS=file:///` to override any inherited HDFS default.

4.5.3 CLI Integration with HDFS

HDFS is now a hard dependency of the CLI pipeline; there is no bind-mount fallback. The `check_hdfs()` function (step 3 of the CLI pipeline) verifies at startup that the `namenode` container is in running state. If it is not, the CLI exits immediately with guidance on how to start the full cluster.

A note on the data paths is warranted here. The CSV input and the YAML scenario configuration are delivered to the cluster by two entirely different mechanisms:

- The CSV file is delivered to HDFS: the CLI uses `docker cp` to copy it into the `namenode` container as a temporary staging file, then executes `hdfs dfs -put -f` from within that container to transfer the data into HDFS proper (i.e. into the DataNode's block store). The NameNode never holds the data itself; it only records the block locations. The resulting HDFS Uniform Resource Identifier (URI) (`hdfs://namenode:9000/inputs/<filename>`) is passed to the Spark job, which reads the blocks directly from the DataNode.
- The YAML scenario configuration bypasses HDFS entirely. Because the YAML file is a small configuration document (typically under one kilobyte) rather than data, sending it through HDFS would be unnecessary overhead. Instead, the CLI injects its content directly into the `spark-master` container's local filesystem via `docker compose exec` using Python's standard input stream. The YAML file is written to `/tmp/` on the master and passed to the Spark job as a local path argument.

This separation matters for the data-flow diagram: the CSV path involves the full HDFS upload pipeline (host → namenode container → HDFS DataNode), whereas the YAML path is a direct host-to-container injection.

The output artefacts follow a third, simpler path: the Spark driver (the Python process running on `spark-master`) writes the aggregated CSV, scatter plot, and JSON summary to a temporary directory inside the master container. After the Spark job completes, the CLI issues a single `docker cp` command to pull all three files from the container to the host `outputs/` directory. No worker container is involved in output delivery.

4.6 MapReduce Strategy: Rationale for `mapPartitions`

4.6.1 The Payload Parsing Constraint

A central design constraint of the distributed processor is that the `payload` column of every simulation row contains a raw Python-literal string representation of a nested dictionary a format generated by the ASA Python API and not directly parseable by any native Spark Structured Query Language (SQL) function. Strict JSON parsing (`from_json`) fails because ASA payloads use single-quoted string literals, which are valid Python but not valid JSON. This means that payload parsing must occur in Python, and the question of which Spark API to use reduces to the question of how to invoke Python parsing with minimum overhead.

4.6.2 Candidate APIs and Their Limitations

Several Spark primitives are conceptually applicable:

DataFrame `groupBy + agg`. This is the natural choice for structured data and native Spark functions. However, it requires that payload parsing be expressed as a Spark SQL User-Defined Function (UDF), which introduces one Python/JVM serialisation crossing per row. Moreover, `groupBy` triggers a full shuffle of the dataset across the network, which is unnecessary for this workload: the aggregation is local within each simulation run (experiment), not a global join.

`RDD.map`. A row-level Python UDF avoids the schema inference problem but incurs one Python/JVM boundary crossing per row. For a batch of four million rows (a typical Quick-Reaction simulation batch), this results in four million individual pickling operations, each transferring one row from the JVM executor to the Python process and back. The overhead has been consistently measured to be substantial: Amazon Web Services (AWS) prescriptive guidance recommends avoiding row-level Python UDFs in favour of vectorised alternatives wherever possible (Amazon Web Services, 2023).

Pandas/Arrow UDF. Apache Arrow (McKinney, 2016) provides a zero-copy columnar memory format that eliminates the pickling step for numeric and string columns, reducing boundary-crossing overhead by batching rows. However, the bottleneck in this workload is not serialisation but computation: `ast.literal_eval` is an inherently sequential Python parser, and the subsequent nested-dictionary traversal and entity-selector matching require Python control flow that cannot be vectorised into Arrow operations. Consequently, an Arrow UDF would improve serialisation cost but leave the dominant per-row Python computation unchanged.

`RDD.aggregateByKey`. This operator supports map-side combination, which would partially replicate the local-accumulation benefit of `mapPartitions`. However, it requires a preliminary `map` call to emit `(key, row)` pairs adding one extra Python crossing per row to extract the experiment identifier and the `seqFunc`/`combFunc` API is less expressive than a general closure, making it difficult to implement the frame-indexed `(frame, value)` accumulation that the processor requires.

`foreachPartition`. This operator offers the same partitioned invocation semantics as `mapPartitions` but cannot return values to the driver; it is intended for side effects such as writing to external storage. It is therefore unsuitable for collecting partial aggregation dictionaries.

4.6.3 Why was `mapPartitions` chosen?

`RDD.mapPartitions` invokes the Python processing function once per partition rather than once per row. The function receives an iterator over all rows in a partition, processes them in a tight Python loop, and yields a small result (one or two dictionaries, one entry per experiment) when the iteration is complete. This design offers five decisive advantages for this workload:

1. Amortised crossing overhead. With $N = 16$ partitions and $M = 4 \times 10^6$ rows, `mapPartitions` produces $N = 16$ Python/JVM crossings; `.map()` would produce $M = 4 \times 10^6$ a factor of $M/N = 250\,000$ fewer crossings.

2. No shuffle. The partial dictionaries yielded by the map phase are tiny: at most one entry per experiment per partition. The entire reduce phase operates on data that is orders of magnitude smaller than the input.

3. Single closure serialisation per executor. The scenario configuration (entity selectors, field paths, aggregation types) is captured in a Python closure that is serialised once when Spark dispatches the task to each executor, not once per row.

4. Expressive accumulation state. The partition function maintains a `dict` of `(frame, value)` pairs that supports arbitrary frame-aggregation strategies (`max`, `min`, `last`, `first`) without the constraints imposed by the `aggregateByKey` API.

5. Academic precedent. Zaharia et al. (Zaharia *et al.*, 2012) specifically identify stateful partition-level computation as a primary design use case for RDDs, distinguishing it from the element-wise `map` transform.

The choice of `mapPartitions` therefore follows directly from the payload parsing constraint: given that Python computation is mandatory and the per-row cost dominates, the

only strategy that reduces Python invocations without introducing an unnecessary shuffle is `mapPartitions`. This conclusion is independent of future improvements to Arrow serialisation or Spark's Python execution model, because the bottleneck is computation, not transport.

The remote deployment of AeroSOuDA, including the two-component architecture, the SSH/SFTP communication design, and the end-to-end data flow, is described in full detail in Chapter 5.

5 Remote Cluster Deployment

Chapter 4 validated the AeroSOuDA concept through a local cluster prototype running on a single workstation. That prototype demonstrated that the proposed distributed MapReduce pipeline is technically feasible. The natural next step is to extend the deployment model so that the Spark and HDFS cluster runs on a dedicated remote server with more computational resources, while the analyst interacts with it from a lightweight client on their own machine without requiring Docker to be installed locally.

This chapter describes the remote cluster deployment architecture, implemented in the `tg-cluster` repository. The repository is split into two independent components: `cluster/`, the Docker Compose stack that runs on the remote VM, and `client/`, a Python-only CLI that runs on the analyst's machine and communicates with the VM exclusively via SSH and SFTP.

The chapter is organized as follows. Section 5.1 presents the two-component architecture and justifies the design decisions. Section 5.2 documents the VM cluster configuration. Section 5.3 describes the remote client CLI and its eight-step pipeline. Section 5.4 explains the SSH/SFTP communication design. Section 5.5 presents the end-to-end data flow from the analyst's machine to the VM cluster and back.

5.1 Architecture Overview

5.1.1 Motivation for a Two-Component Design

The local cluster prototype from Chapter 4 requires the analyst to have Docker installed and to run the entire Spark and HDFS stack on their own machine. This is acceptable for development and testing but impractical for production use: simulation result CSV files can reach several gigabytes; a local workstation may lack the memory and processing cores to distribute such heavy workload effectively; and the cluster must be shut down and restarted each time the analyst switches computers.

A remote cluster hosted on a VM solves all three problems: the VM can be provisioned

with many cores and ample memory, it stores the simulation CSV files persistently on HDFS, and it is accessible from any machine with an internet connection. The analyst only needs Python and a handful of pure-Python dependencies on their laptop in order to run this repository. This project also includes in its `README` file a step-by-step guide to install said python dependencies.

5.1.2 Repository Structure

The `tg-cluster` repository is organized into two top-level directories:

- `cluster/`, the Docker Compose stack intended to be deployed on the remote VM. It contains the same Spark and HDFS containers as the local cluster, together with the processing scripts, scenario YAML files, and all configuration files. Analysts clone this directory on the VM and start it with `make up`.
- `client/`, the analyst-side CLI. It has no Docker dependency; its only requirements are Python 3.8+ and the packages listed in `requirements.txt` (rich, questionary, pyyaml, pandas, paramiko). The analyst runs `python cli.py` on their laptop to initiate the full distributed pipeline on the remote VM.

Figure 5.1 illustrates the two-component architecture and the communication channels between them.

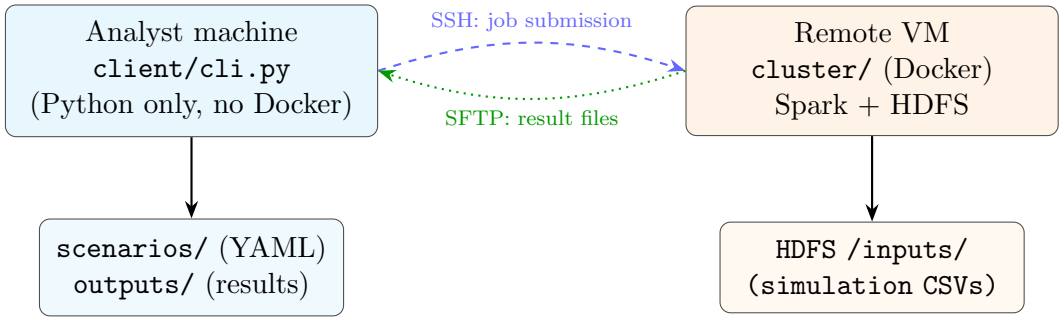


FIGURE 5.1 – Two-component architecture of the AeroSOuDA remote deployment. Dashed blue arrows indicate SSH connections; dotted green arrows indicate SFTP file transfers. YAML scenario files reside on the analyst's machine; simulation CSV files reside on the VM's HDFS.

5.2 The VM Cluster (`cluster/`)

5.2.1 Container Configuration

The `cluster/docker-compose.yml` defines the same thirteen-container topology as the local cluster: one HDFS NameNode, four HDFS DataNodes, one Spark master, six

Spark workers, and one Spark History Server.

Whereas the local cluster is started by the analyst on their own workstation and torn down after each session, the VM cluster is intended to run continuously. Simulation result CSV files are uploaded to the HDFS storage layer once and remain there for repeated analysis runs.

5.2.2 Persistent HDFS Storage

One of the main advantages of the remote deployment is that large CSV files are uploaded to the VM's HDFS once and do not need to be retransferred for each analysis run. The simulation result files can be copied to the VM via the Secure Copy Protocol (`scp`) (or a similar file transfer tool), then loaded into HDFS with:

```
docker cp results.csv namenode:/tmp/results.csv
docker exec namenode hdfs dfs -put /tmp/results.csv /inputs/
```

After this one-time upload, any number of analysis runs with different YAML scenario configurations can be performed without retransferring the CSV.

5.2.3 VM Setup

Deploying the cluster on a fresh VM requires only Docker and Docker Compose. The setup procedure is documented in `cluster/README.md` and consists of three steps:

1. Install Docker Engine and the Compose plugin following the official Docker documentation.
2. Clone the `tg-cluster` repository and navigate to `cluster/`.
3. Run `make fix-perms && make up` to fix volume permissions and start all containers.

The HDFS web interface is available at `http://<VM_IP>:9870` and the Spark web interface at `http://<VM_IP>:8080`. Both interfaces should be accessible from the analyst's machine if the VM firewall allows Transmission Control Protocol (TCP) connections on those ports.

5.3 The Remote Client (`client/`)

5.3.1 Overview

The remote client (`client/cli.py`) is the analyst-facing entry point for the remote cluster deployment. It provides the same interactive experience as the local CLI but replaces all Docker commands with SSH and SFTP operations directed at the remote VM. The client has no Docker dependency; it uses the `paramiko` library (Paramiko contributors, 2024) for SSH connectivity and the standard `pyyaml` and `rich` libraries for scenario management and terminal output.

5.3.2 Pipeline Flow

Figure 5.2 presents the remote client pipeline. The pipeline is composed of eight sequential stages.

The stages of the pipeline are described as follows:

1. Welcome screen. A styled panel introduces the tool, identifies the remote-deployment context, and lists the pipeline steps.

2. VM connection prompt. The client checks for a `.env` file in the working directory containing the five connection variables: `SSH_IP`, `SSH_USERNAME`, `SSH_PASSWORD`, `SSH_PORT`, and `VM_CLUSTER_PATH`. If the file is present and all variables are set, all connection parameters are loaded automatically and no prompt is shown. Otherwise, the analyst is prompted interactively for the VM Internet Protocol (IP) address, SSH username, authentication method (password or private key file path), port, and cluster directory path. The `.env` approach is the recommended workflow for repeated use against the same cluster, as it eliminates repeated manual input.

3. SSH connection. A `paramiko.SSHClient` is opened to the VM. The connection module then issues a `docker ps` command over SSH to verify that the cluster containers (`spark-master`, `namenode`, DataNodes, and workers) are running. If any required container is absent, the pipeline exits with actionable guidance.

4. CSV listing from HDFS. The client issues the command `docker exec namenode hdfs dfs -ls /inputs/` over SSH and parses the output to build a list of available CSV files together with their sizes. This listing reflects the current contents of the HDFS storage layer on the VM.

5. CSV and scenario selection. The analyst selects one of the listed CSV files and either loads a YAML scenario from the local `scenarios/` directory or creates a new one using the same guided wizard as the local CLI.

6. Execution confirmation. A summary panel displays the selected CSV name, the VM host, and the scenario configuration. The analyst confirms before the job is submitted.

7. Job submission and monitoring. The client performs the following sub-steps:

   a. SFTP: upload the local YAML scenario to `/tmp/` on the VM.

   b. SSH: inject the YAML into the `spark-master` container via `docker exec`.

   c. SSH: execute `run_csv_processor.sh` inside `spark-master`, streaming `stdout` back through the SSH channel. Spark stage-progress lines are parsed and displayed as a live progress bar on the analyst's terminal.

   d. SSH: copy output artefacts from the container to a temporary directory on the VM via `docker cp`.

8. Result transfer. The client uses SFTP to download all output files from the VM's temporary directory to the local `outputs/` directory. The three output artefacts an aggregated CSV, a scatter plot image, and a JSON summary are saved locally on the analyst's machine.

9. Results display. A final panel reports the scenario name, the number of data points, the batch aggregate value, and the local paths of the output files.

5.4 SSH and SFTP Communication Design

5.4.1 The `RemoteCluster` Class

All SSH and SFTP operations are encapsulated in the `cli/remote.py::RemoteCluster` class. On `connect()`, it opens a single `paramiko.SSHClient` connection to the VM and derives an `SFTPClient` from the same underlying transport, minimizing the number of TCP handshakes.

For job execution, `run_job()` opens a pseudo-terminal channel on the existing transport. The pseudo-terminal is required because the `docker compose exec` command on the VM buffers output differently when it detects that its standard output is not a Teletypewriter (terminal device) (TTY). Allocating a pseudo-terminal forces line-buffered output, which is necessary for the stage-progress lines to appear in real time on the analyst's terminal.

5.4.2 Authentication

The `RemoteCluster` constructor accepts either a password or the path to an SSH private key file. When a key path is provided, `paramiko` loads the key with `paramiko.RSAKey.from_private_key_file()`. Both authentication methods are equivalent in security from `paramiko`'s perspective; using a private key is recommended for production deployments because it avoids storing or transmitting passwords.

5.4.3 Data Transfer Design

Three distinct data-transfer patterns are used in the pipeline:

1. YAML upload (SFTP PUT): the analyst's local YAML scenario file is uploaded to `/tmp/aerosouda_scenario.yaml` on the VM filesystem, then injected into the `spark-master` container via a Python one-liner piped through `docker exec -i`.
2. Job output export (SSH command): after the Spark job completes, the output artefacts are copied from the container's temporary directory to a staging area on the VM filesystem via `docker cp`.
3. Result download (SFTP GET): the SFTP client iterates over the staging directory and downloads each file to the local `outputs/` directory.

This design avoids mounting any host directories into the Docker containers on the VM: all file movements cross container boundaries via `docker cp`, and all client–VM movements use SFTP. The CSV files themselves are never transferred from the VM to the client; only the small result artefacts (aggregated CSV, scatter plot, JSON summary) are downloaded.

5.5 End-to-End Data Flow

Figure 5.3 presents the complete end-to-end data flow of the remote deployment from the analyst's perspective.

The key insight of the design is the asymmetry between the upward and downward data flows. In the upward direction (analyst → VM), only the small YAML scenario file (typically a few kilobytes) is transferred. The large simulation CSV files are already resident on the VM's HDFS storage layer, uploaded once by a VM operator. In the downward direction (VM → analyst), only the compact result artefacts are downloaded: an aggregated CSV (at most a few hundred rows), a scatter plot image, and a JSON

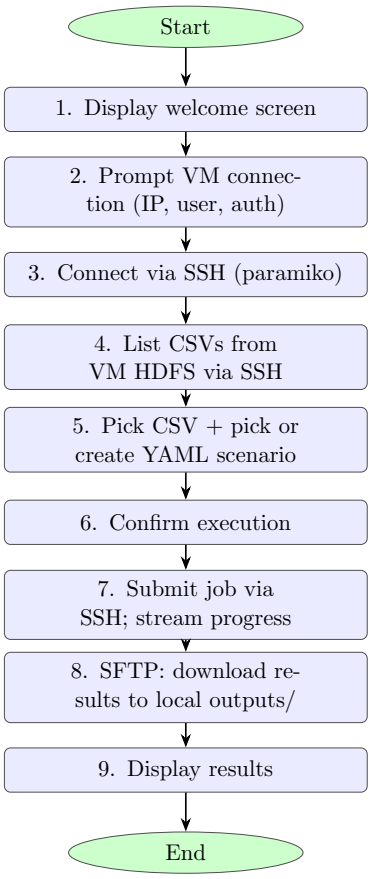


FIGURE 5.2 – Execution pipeline of the AeroSOuDA remote client (`client/cli.py`).

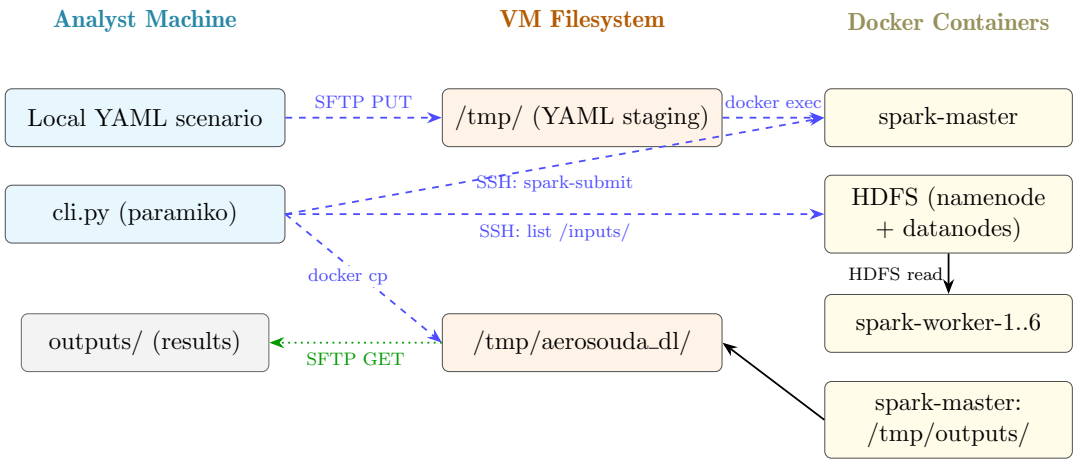


FIGURE 5.3 – End-to-end data flow of the AeroSOuDA remote deployment. Dashed blue arrows indicate SSH-mediated commands; dotted green arrows indicate SFTP file transfers; solid black arrows indicate direct data reads within the Docker network.

summary. This design minimises network bandwidth consumption and makes the pipeline practical even on a slow connection.

The simulation CSV data never leaves the VM. It is ingested by the Spark workers directly from HDFS within the Docker bridge network, processed in memory across the six worker containers, and the aggregated results are written to the master's temporary directory. Only the aggregated output is exfiltrated to the analyst's machine.

5.5.1 Separation of Concerns

The two-repository design enforces a clean separation of concerns:

- Cluster (`tg-cluster/cluster/`): responsible for storage and computation. It manages the HDFS data layer and the Spark execution engine. Its only external interface is a standard Docker Compose stack accessible via SSH.
- Client (`tg-cluster/client/`): responsible for user interaction and orchestration. It knows the analyst's intent (which CSV, which scenario) and coordinates all actions on the VM cluster via SSH commands. It has no knowledge of Spark internals or HDFS block placement.

This separation means that the cluster can be upgraded (more workers, larger DataNodes, different Spark version) without any changes to the client, and the client can be updated (new scenario wizard features, different output formats) without touching the cluster configuration.

5.5.2 Execution Phase Instrumentation

The `scripts/processor/spark_job.py` module emits fine-grained timing markers at key boundaries of the pipeline. Each marker is printed to `stdout` with the prefix `METRIC:`, for example:

```
METRIC:T_INIT_SECONDS=18.432160
METRIC:T_MAP_SECONDS=55.117832
METRIC:T_REDUCE_SECONDS=0.009412
METRIC:T_OUTPUT_SECONDS=0.421073
```

The four markers decompose T_{job} into four sequential phases:

- T_{init}: time from `SparkSession.getOrCreate()` returning to the first data-read call. Captures JVM initialisation, Spark context setup, and worker registration.

- T_{map}: time for the `mapPartitions().collect()` action. This is the dominant phase and encompasses both HDFS data reads (I/O) and distributed Python payload parsing (computation). Because Spark evaluates plans lazily, these two sub-phases cannot be separated without an additional materialise step (e.g., `cache().count()`) that would change the benchmark semantics.

- T_{reduce}: time for the driver-side reduce loop that merges the per-partition partial results into the global accumulators. This phase is $O(n_{\text{entities}})$ and is expected to be negligible.

- T_{output}: time for constructing the output Pandas structure, writing the aggregated CSV, rendering the scatter plot, and serialising the JSON summary.

The `tg-cluster` benchmark harness parses these markers in real time as the SSH output stream arrives and records them as additional columns in the benchmark CSV (`t_init_s`, `t_map_s`, `t_reduce_s`, `t_output_s`). In parallel, a background sampling thread issues periodic `docker stats` queries over the same SSH connection to record peak and mean worker Central Processing Unit (CPU) usage (`cpu_peak_pct`, `cpu_mean_pct`) and peak memory consumption (`mem_peak_mb`) across the four Spark worker containers.

This instrumentation provides the empirical basis for quantifying host-resource contention at scale and provides enough content to test and analyse this thesis' capacities in the next session.

6 Experimental Evaluation

6.1 Overview

This chapter presents the experimental evaluation of the AeroSOuDA Remote cluster prototype described in Chapter 5. The primary objective is to quantify the performance benefit of the distributed MapReduce pipeline over a single-executor baseline on the same Spark infrastructure for the representative production-scale simulation batch used in the study.

The evaluation is structured as a controlled benchmarking study. Section 6.1 defines the experimental setup: the input datasets, analysis scenarios, cluster configurations, and measurement protocol. Section 6.2 introduces the performance metrics collected. The results for each experimental condition are presented in Sections 6.3 and 6.4, followed by a discussion of speedup, overhead, and scalability in Section 6.5.

6.2 Experimental Setup

6.2.1 Input Dataset

The benchmark uses two simulation batch CSV datasets:

- Quick-Reaction dataset: 1 888 850 rows, approximately 1.3 GB, involving 10 simulation runs. This is the production batch generated from the Quick-Reaction scenario described in Section 2.4.
- Launcher Longitude dataset: 2 290 288 rows, approximately 1.5 GB, also involving 10 simulation runs. This batch consists in varying the longitude of a Ground-Based Air Defence (GBAD) missile launcher across those 10 runs. This dataset's scenario works the same way as the QR scenario does, only changing the fighters for the GBAD launcher as the opposing force.

6.2.2 Analysis Scenarios

Two YAML scenario configurations are evaluated in this chapter:

1. Quick-Reaction: computes the Pearson correlation between the longitude of the red-side launcher and the survival time (maximum frame number) of blue-side fighter entities. This scenario uses `scope: per_entity` for the Y variable, producing one data point per (entity, experiment) pair.

2. Launcher Longitude: computes the Pearson correlation between the longitude of the red-side GBAD launcher and the maximum damage sustained by blue-side fighter entities. The aggregation field (`state.damage`) requires deeper nested-dictionary traversal than the frame-counting in Scenario 1.

Both scenarios use `scope: per_experiment` for the X variable and `scope: per_entity` for the Y variable. Consequently, one experiment-level X value can be paired with multiple entity-level Y observations (two blue fighters per run). Pearson correlation remains well-defined for these observations, but rows from the same experiment are not strictly independent; results are therefore interpreted as an entity-level association measure conditioned on the experiment-level parameter setting.

6.2.3 Processing Modes: Serial and Parallel

A central methodological question is how to define a serial baseline that is comparable to the parallel implementation without requiring a separate codebase. This work adopts the following definition:

- Serial mode (baseline): the distributed processor runs with a single executor, single core, and single RDD partition (`executor.instances=1`, `executor.cores=1`, `-agg-partitions 1`). The code path is identical to the parallel mode; only the resource configuration differs.

- Parallel mode (target): the processor uses four executors, with each executor assigned two cores and the RDD partitioned into 16 segments (default configuration). Four executors execute map tasks concurrently.

This baseline definition ensures an apples-to-apples comparison: both modes use the same Spark infrastructure, the same code, and the same HDFS storage layer. The measured speedup therefore reflects the benefit of task-level parallelism within the AeroSOuDA architecture, including all fixed costs (JVM startup, HDFS read, output serialisation).

Using a single Spark executor as the serial baseline also avoids conflating the inherent overhead of the Spark framework with the overhead attributable to parallelism. A comparison against a standalone Python pandas script while informative would introduce confounding factors such as different memory access patterns and the absence of Spark's partitioned data model.

6.2.4 Measurement Protocol

Each job execution follows the same protocol, illustrated in the Benchmark Workflow in Figure 6.1:

1. The input CSV is uploaded to HDFS before the timing window begins.
2. The Spark job is launched remotely via SSH by the `tg-cluster` benchmark harness, which executes `run_csv_processor.sh` (or `run_csv_processor_serial.sh`) inside the `spark-master` container.
3. Two timestamps are recorded: t_0 (spark-submit invoked on the remote host) and t_1 (job process exits on the remote host), giving $T_{\text{job}} = t_1 - t_0$. Additionally, the wall-clock time from the local client perspective (T_{total}, including SSH round-trip overhead) is recorded. The benchmark harness also parses fine-grained phase timing markers emitted by the job (T_{init}, T_{map}, T_{reduce}, T_{output}) and records peak worker CPU and memory via periodic `docker stats` queries; the instrumentation is described in Section 5.5.2.
4. The process is repeated ten times per mode. All ten runs per mode are retained for statistical analysis (see Section 6.4).

6.3 Performance Metrics

Core metrics are collected for each job execution:

- Total job time (T_{job}, seconds): wall-clock time measured on the remote host from the moment `spark-submit` is invoked until the job process exits. This metric captures the full end-to-end runtime including Spark startup, HDFS read, computation, reduce phase, and output write.
- Total client time (T_{total}, seconds): wall-clock time as seen by the local benchmark client, encompassing T_{job} plus the SSH command round-trip overhead. The difference $\Delta_{\text{ssh}} = T_{\text{total}} - T_{\text{job}}$ reflects network and shell invocation latency and is expected to be smal overall.

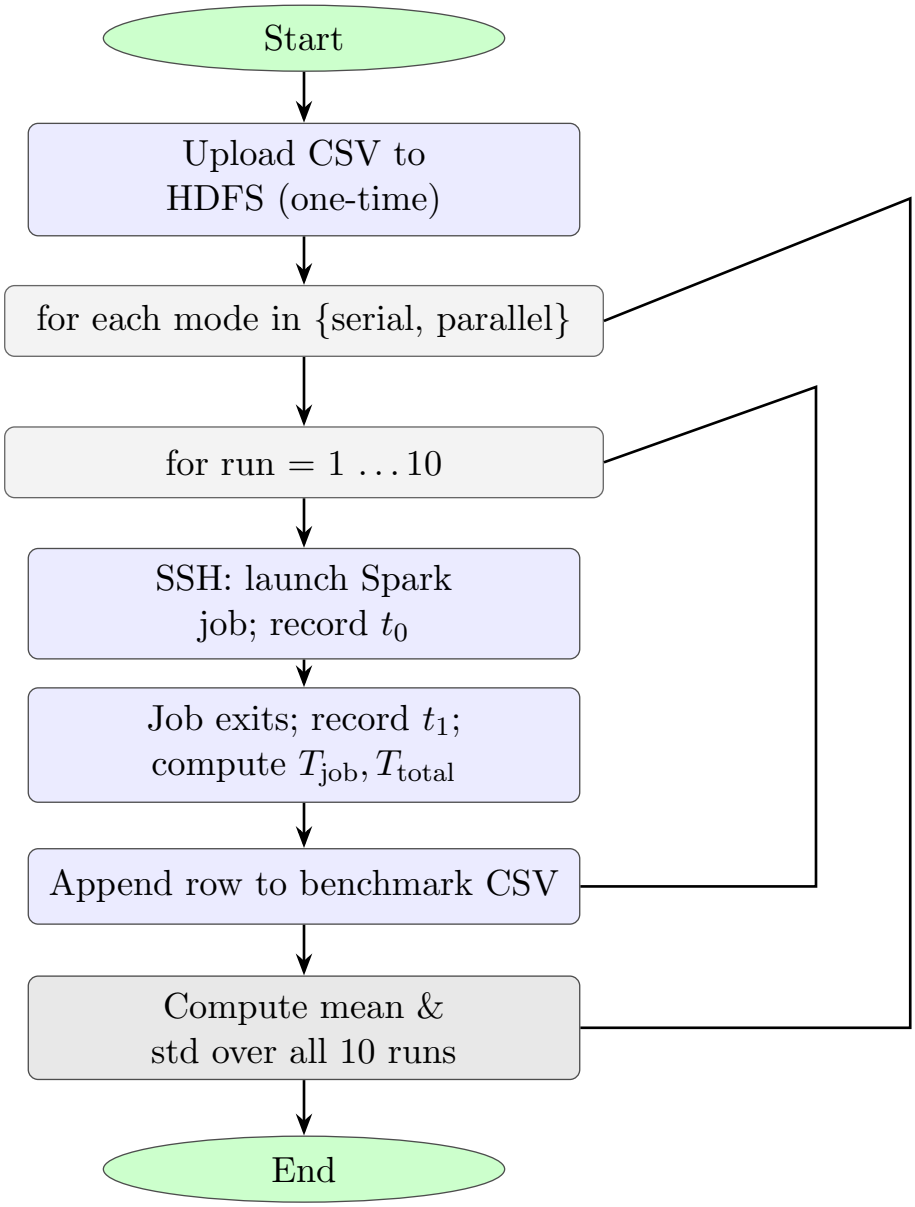


FIGURE 6.1 – Benchmark measurement protocol. The outer loop iterates over the two processing modes; the inner loop repeats the job ten times per mode. All ten runs per mode are retained after a post-hoc warmup check (Section 6.4) found no discardable outlier.

- Throughput ($R = N_{\text{rows}}/T_{\text{job}}$, $\text{rows}\,\text{s}^{-1}$): number of simulation rows processed per second. Allows comparison across datasets of different sizes.
- Speedup ($S = \bar{T}_{\text{job}}^{\text{serial}}/\bar{T}_{\text{job}}^{\text{parallel}}$, dimensionless): ratio of the mean serial job time to the mean parallel job time. The theoretical maximum speedup with $N = 4$ executors under Amdahl's law (Amdahl, 1967) is $S_{\max} = 1/(f + (1 - f)/N)$, where f is the non-parallelisable fraction.
- Parallel efficiency ($\eta = S/N$, dimensionless): fraction of the theoretical linear speedup achieved in practice.

In addition to these core metrics, the benchmark harness records fine-grained phase timings emitted by the remote Spark job:

$$T_{\text{init}},\ T_{\text{map}},\ T_{\text{reduce}},\ T_{\text{output}}.$$

Given Spark's lazy execution model, T_{map} includes both HDFS read I/O and distributed payload processing. A residual term is also computed,

$$T_{\text{other}} = T_{\text{job}} - (T_{\text{init}} + T_{\text{map}} + T_{\text{reduce}} + T_{\text{output}})\,,$$

to capture fixed runtime costs not directly attributed to the mapped phases (for example, job orchestration and scheduler overhead). Finally, peak worker CPU and memory are sampled via `docker stats` throughout each run.

6.4 JVM Warm-up Analysis

Before computing summary statistics, a post-hoc outlier check was performed on the first run of each mode to test whether JVM and Just-In-Time (JIT) warm-up effects distort the initial measurements. Warm-up would manifest as run 1 being noticeably slower than the remaining runs.

Table 6.1 compares the run-1 job time against the mean of runs 2–10 for each mode. For the parallel mode, run 1 (76.10 s) is 0.77 s above the subsequent mean (75.33 s). For the serial mode, run 1 (213.30 s) is actually faster by 2.25 s than the subsequent mean (215.55 s).

No warm-up artefact is detected: neither run 1 is a statistical outlier, and the serial first run is directionally faster, not slower. Accordingly, all ten runs per mode are retained for computing summary statistics.

TABLE 6.1 – JVM warm-up check: run 1 vs. mean of runs 2–10.

Mode	Run 1 T_{job} (s)	Mean runs 2–10 (s)
Parallel	76.10	75.33
Serial	213.30	215.55

6.5 Results

6.5.1 Quick-Reaction Scenario

Executed campaign. The benchmark was executed using the `tg-cluster` remote client (Chapter 5) connected via SSH to the AeroSOuDA cluster. Input: `results_quickreaction.csv` (1 888 850 rows, ≈1.3 GB) with scenario `qr_correlation.yaml` (Pearson correlation, Quick-Reaction Scenario). Two modes were measured: parallel (4 executors, default configuration) and serial (1 executor, 1 core, 1 partition as baseline). Each mode was repeated ten times; all ten runs are used for statistics (see Section 6.4).

Tables 6.2 and 6.3 list the per-run measurements. Table 6.4 summarises the aggregate statistics.

TABLE 6.2 – Quick-Reaction, per-run timings (serial mode, 1 executor).

Run	T_{job} (s)	T_{total} (s)	R (rows/s)
1	213.30	213.66	8852.5
2	216.29	216.63	8730.3
3	216.80	217.15	8709.8
4	216.78	217.15	8710.6
5	214.26	214.67	8813.1
6	215.72	216.06	8752.4
7	215.78	216.12	8750.0
8	215.79	216.12	8749.9
9	213.78	214.16	8832.6
10	214.78	218.37	8792.1

6.5.2 Launcher Longitude Scenario

Executed campaign. The Launcher Longitude benchmark was executed using the standard main-benchmark protocol: ten serial runs (1 executor) and ten parallel runs (4 executors). Input: `results_gbad_launcher_longitude.csv` (2 290 288 rows, ≈1.5 GB) with scenario `analyze_fighter_damage_by_launcher_longitude.yaml` (Pearson correlation between GBAD launcher longitude and the damage taken by the fighter jets). Both modes were repeated ten times; all ten runs are used for statistics.

TABLE 6.3 – Quick-Reaction, per-run timings (parallel mode, 4 executors).

Run	T_{job} (s)	T_{total} (s)	R (rows/s)
1	76.10	76.48	24820.6
2	74.62	74.94	25312.9
3	76.10	76.44	24820.6
4	75.10	75.50	25151.1
5	75.60	75.95	24984.8
6	74.59	74.93	25323.1
7	75.60	75.94	24984.8
8	75.61	75.98	24981.5
9	75.61	75.94	24981.5
10	75.17	75.51	25128.3

TABLE 6.4 – Quick-Reaction, summary statistics (all 10 runs per mode).

Metric	Serial (mean ± std)	Parallel (mean ± std)
T_{job} (s)	215.33 ± 1.24	75.41 ± 0.53
T_{total} (s)	216.01 ± 1.47	75.76 ± 0.54
Δ_{ssh} (s)	0.68 ± 1.02	0.35 ± 0.03
R (rows s^{-1})	$8\,769 \pm 50$	$25\,049 \pm 177$
Derived metrics		
Speedup S	2.855	
Parallel efficiency η	0.714 (71.4%)	

Timing results. Tables 6.5 and 6.6 list the per-run measurements. Table 6.7 summarises the aggregate statistics.

TABLE 6.5 – Launcher Longitude, per-run timings (serial mode, 1 executor).

Run	T_{job} (s)	T_{total} (s)	R (rows/s)
1	250.81	251.65	9 132
2	252.81	253.58	9 059
3	253.82	254.64	9 023
4	251.81	252.65	9 095
5	250.30	251.08	9 150
6	252.26	253.08	9 079
7	252.83	253.69	9 059
8	252.29	253.09	9 078
9	250.23	251.05	9 153
10	253.24	254.13	9 044

TABLE 6.6 – Launcher Longitude, per-run timings (parallel mode, 4 executors).

Run	T_{job} (s)	T_{total} (s)	R (rows/s)
1	81.61	82.45	28 064
2	79.60	80.47	28 772
3	80.11	80.92	28 589
4	79.59	80.42	28 776
5	80.10	81.01	28 593
6	79.61	80.44	28 769
7	81.11	80.96	28 237
8	80.11	80.96	28 589
9	79.61	80.47	28 769
10	80.11	80.91	28 589

The Launcher Longitude scenario processes a larger dataset than Quick-Reaction (2.29M vs 1.89M rows) and achieves a higher speedup (3.144 vs 2.855) and parallel efficiency (78.6% vs 71.4%). This is consistent with Amdahl-type behaviour: a larger parallelisable workload reduces the relative weight of the non-parallelisable serial component (Spark session startup, driver initialisation, output serialisation), thereby yielding a higher observed speedup. Section 6.7 characterises this behaviour in detail using a scalability experiment over the same input file.

6.6 Discussion

6.6.1 Speedup Analysis

Table 6.8 summarises the speedup and efficiency achieved by each campaign.

TABLE 6.7 – Launcher Longitude, summary statistics (all 10 runs per mode).

Metric	Serial (mean ± std)	Parallel (mean ± std)
T_{job} (s)	252.04 ± 1.24	80.16 ± 0.69
T_{total} (s)	252.86 ± 1.21	80.90 ± 0.57
Δ_{ssh} (s)	0.82 ± 0.04	0.75 ± 0.32
R (rows s^{-1})	$9\,087 \pm 42$	$28\,573 \pm 243$
Derived metrics		
Speedup S	3.144	
Parallel efficiency η	0.786 (78.6%)	

TABLE 6.8 – Speedup and efficiency comparison across the two benchmark campaigns.

Campaign	Dataset rows	$\bar{T}_{\text{serial}}$ (s)	S	η
Quick-Reaction	1 888 850	215.33	2.855	71.4%
Launcher Longitude	2 290 288	252.04	3.144	78.6%

Both campaigns achieve meaningful parallel acceleration on a single-host Docker deployment. The higher speedup and efficiency of the Launcher Longitude campaign ($S = 3.144$, $\eta = 78.6\%$) relative to Quick-Reaction ($S = 2.855$, $\eta = 71.4\%$) is consistent with the larger input volume: the additional $\approx 400\,000$ rows increase the parallelisable fraction of the job, reducing the relative overhead of the fixed serial components and allowing the four executors to deliver a higher net acceleration. The key take-away is that the AeroSOuDA distributed pipeline reduces wall-clock time by 65–75% relative to the single-executor baseline, independently of which analytical scenario or dataset is used.

6.6.2 Amdahl's Law and Overhead Analysis

Amdahl's law (Amdahl, 1967) predicts that for a programme with non-parallelisable fraction f, the speedup with N executors is $S(N) = 1/(f + (1 - f)/N)$.

In this work, f is estimated empirically from the Launcher Longitude scalability timings (Section 6.7) by fitting $T(N) = T_s + T_p/N$, where T_s is the serial component and T_p is the parallelisable component. Using $N \in \{1, 2, 4\}$, the fit yields $\hat{f} = T_s/(T_s + T_p) \approx 0.077$, which corresponds to $S_{\max}(2) \approx 1.857$ and $S_{\max}(4) \approx 3.247$. These values are close to the measured speedups (1.867 and 3.232, respectively), confirming that the observed scaling is well-described by Amdahl-type behaviour under shared-host contention.

6.6.3 Throughput Scaling

Table 6.9 summarises throughput for both campaigns. For Quick-Reaction, throughput increased from 8 769±50 rows/s to 25 049±177 rows/s, a factor of 2.86×. For Launcher Longitude, it increased from 9 087 ± 42 rows/s to 28 573 ± 243 rows/s, a factor of 3.14×, matching the respective speedup values.

TABLE 6.9 – Throughput comparison across both benchmark campaigns (all 10 runs per mode).

Campaign	Mode	Mean (rows/s)	Std (rows/s)
Quick-Reaction	Serial	8 769	50
	Parallel	25 049	177
Launcher Longitude	Serial	9 087	42
	Parallel	28 573	243

6.6.4 SSH Communication Overhead

The difference $\Delta_{\text{ssh}} = T_{\text{total}} - T_{\text{job}}$ captures the round-trip cost of the SSH command dispatch used by the `tg-cluster` client. For the Quick-Reaction parallel mode, the mean overhead is 0.35 ± 0.03 s less than 0.5% of total job time. For the Launcher Longitude parallel mode, the mean overhead is 0.75 ± 0.32 s, less than 1% of total job time. In both cases, SSH dispatch adds negligible latency for the distributed execution model.

6.6.5 Threats to Validity

A couple of factors limit the generalisability of these results:

- Single-host deployment. All containers (master, four workers, namenode, datanodes, history server) ran on the same physical machine. The HDFS data-locality benefit, a primary architectural advantage cannot be realised in this configuration. On a genuine multi-node cluster, network-aware data placement would reduce inter-node I/O and is expected to improve speedup further.
- Shared system resources. Worker containers compete for the same physical CPU cores, memory, and disk I/O, introducing contention that would not exist on dedicated hardware and limits efficiency at higher executor counts. The phase-timing instrumentation described in Section 5.5.2 enables future benchmark runs to quantify each phase's contribution to T_{job} and to correlate worker CPU saturation (measured via `docker stats`) with the observed efficiency drop from $\eta = 93.4\%$ at $N = 2$ to $\eta = 80.8\%$ at $N = 4$, providing an empirical basis for the shared-resource hypothesis.

- Dataset coverage. Two datasets were evaluated: the Quick-Reaction batch (1.3 GB, 1.89M rows) and the Launcher Longitude batch (1.5 GB, 2.29M rows), both consisting in only 10 simulations due to input gathering limitations. Additional campaigns with broader data volumes and different entity schemas would strongly help generalizing those results.

6.7 Scalability Analysis

6.7.1 Motivation

The main benchmark of Section 6.5 provides two data points on the speedup curve: N=1 (serial baseline) and N=4 (parallel mode). A single pair of points is sufficient to claim that parallelism is beneficial, but it does not reveal whether the system scales efficiently as the number of executors grows. This section closes that gap by running the Launcher Longitude full-dataset campaign with N=1, 2, and 4 executors and comparing the resulting speedup curve against the Amdahl's law prediction.

6.7.2 Experimental Setup

The scalability experiment uses the Launcher Longitude input file (`results_gbad_launcher_longitude.csv`, `analyze_fighter_damage_by_launcher_longitude.yaml`). Three executor configurations are evaluated:

- N=1: 1 executor, 1 core, 1 partition.
- N=2: 2 executors, 2 cores maximum.
- N=4: 4 executors, 4 cores maximum.

Each configuration is executed with the `tg-cluster` benchmark harness using the `-mode scalability` flag:

```
python benchmark.py --runs 5 --mode scalability \
    --csv results_gbad_launcher_longitude.csv --no-download
```

Five runs per configuration are used. One run at N=2 produced an execution time of 882.65 s, an outlier attributable to a transient cluster scheduling hiccup and was excluded; the remaining four N=2 runs were used for statistics.

6.7.3 Theoretical Prediction

Using the fitted non-parallelisable fraction $\hat{f} \approx 0.077$ estimated from the scalability timings (Section 6.6.2), Amdahl's law predicts:

$$S_{\max}(N) = \frac{1}{\hat{f} + (1 - \hat{f})/N} \approx \frac{1}{0.077 + 0.923/N}.$$

TABLE 6.10 – Amdahl's law predictions vs. measured speedup for the scalability experiment (Launcher Longitude dataset).

N	$S_{\max}$ (Amdahl)	S (measured)	η (measured)
1	1.000	1.000	—
2	1.857	1.867	93.4%
4	3.247	3.232	80.8%

6.7.4 Results

TABLE 6.11 – Scalability experiment, mean T_{job} and speedup by executor count (5 runs per configuration, Launcher Longitude dataset).

N	$\bar{T}_{\text{job}}$ (s)	σ (s)	Speedup S	Efficiency η
1	260.55	4.39	1.000	—
2	139.55	0.75	1.867	93.4%
4	80.61	0.87	3.232	80.8%

The N=4 result is consistent with the 10-run main benchmark (Table 6.7), confirming measurement stability across independent sessions. The N=1 mean (260.55 s) is slightly higher than the main benchmark serial mean (252.04 s), which is expected as the scalability campaign ran under different system-load conditions.

6.7.5 Discussion

The results in Table 6.10 exhibit a clear and expected trend: speedup increases with N but at a diminishing rate, consistent with Amdahl's law. Two observations are particularly relevant for this thesis.

High efficiency at N=2. With two executors, the system achieves $S = 1.867$ and $\eta = 93.4\%$, very close to the fitted Amdahl prediction ($S_{\max}(2) = 1.857$). At this scale, the shared-host deployment introduces limited contention and the job is nearly perfectly parallel.

Efficiency drop from N=2 to N=4. The efficiency falls from 93.4% to 80.8% as the executor count doubles. This degradation is consistent with shared-host contention: all four worker containers compete for the same physical cores, memory bus, and disk I/O. Even so, quadrupling the executor count reduces wall-clock time by a factor of 3.23×, achieving the highest speedup measured in this thesis.

Cross-dataset consistency. The Quick-Reaction campaign (Section 6.5.1) uses a smaller dataset (1.89M rows) and shows a measured speedup of 2.855 at N=4 with 71.4% efficiency. The Launcher Longitude campaign uses a larger dataset (2.29M rows) and achieves 3.144 at N=4 from the main benchmark and 3.232 from the scalability experiment. The increase in speedup with dataset size is consistent with Amdahl's law: a larger parallelisable workload reduces the relative serial overhead fraction, raising the efficiency ceiling. Taken together, the two campaigns confirm that the distributed pipeline scales beneficially across different input volumes and analytical configurations.

7 Conclusions

This chapter synthesises the contributions of the work, discusses the experimental findings in the broader context of the motivation established in Chapter 1, and identifies the most relevant directions for future research.

7.1 Summary of Contributions

This work proposed, implemented, and evaluated AeroSOuDA (Aerospace Simulation Output Distributed Architecture), a distributed computing architecture designed to accelerate the post-processing of aerospace simulation outputs in the context of the ASA platform.

The central problem addressed was the analytical bottleneck that arises when large simulation batches, each batch potentially comprising millions of entity-state records, are processed serially on a single machine. The existing ASA workflow requires the analyst to retrieve raw outputs centrally, compute metrics in sequence, and manually produce aggregate values, a workflow that becomes progressively less tractable as batch size grows.

AeroSOuDA addresses this bottleneck through three architectural principles:

1. Distributed storage. Simulation result CSV files are ingested into HDFS, where they are partitioned and replicated across multiple DataNode containers. This eliminates the single-machine storage bottleneck and enables data-local computation.

2. Parallel metric extraction. A PySpark job, following the MapReduce paradigm, processes the HDFS-resident data in parallel across multiple executor containers. Each executor independently extracts entity-level metrics from its assigned data partitions (map phase), and the driver merges the partial results into a batch-level aggregate value (reduce phase).

3. Analyst-facing interface. An interactive CLI allows the analyst to define or select a scenario configuration, specifying the metric of interest and the aggregate function,

before job submission, without requiring knowledge of Spark internals or HDFS administration.

Two deployment modes were implemented and validated. The local cluster mode (Chapter 4) runs the entire Spark and HDFS stack on the analyst's workstation via Docker Compose, providing a self-contained development and testing environment. The remote cluster mode (Chapter 5) moves the compute-intensive processing to a dedicated virtual machine, while the analyst interacts through a lightweight Python client using only SSH and SFTP as ways to communicate with the remote machine. This design ensures that large simulation datasets never need to be transferred to the analyst's machine before any analytical reduction has occurred.

7.2 Results Discussion

The experimental evaluation presented in Chapter 6 measured the performance of the distributed pipeline on three analytical scenarios over two distinct batch datasets.

Speedup and efficiency. The parallel configuration (four executors) reduced the mean job time from $215.33 \pm 1.24\,\text{s}$ to $75.41 \pm 0.53\,\text{s}$, yielding a speedup of $S = 2.855$ and a parallel efficiency of $\eta = 71.4\%$. An efficiency between 71% and 79% on a single-host Docker deployment, where all containers compete for the same physical CPU cores, memory bus, and disk I/O, represents a strong baseline result. On dedicated multi-node hardware, where this contention is absent, further efficiency gains are plausible and should be quantified in future multi-node experiments.

Throughput. The distributed pipeline processes $25\,049 \pm 177$ rows per second, compared to $8\,769 \pm 50$ rows per second for the single-executor baseline, a $2.86\times$ improvement. Crucially, only the compact analytical results (an aggregated CSV, a scatter plot, and a JSON summary) are delivered to the analyst, while the full 1.3 GB dataset remains on the VM's HDFS.

Scalability. The scalability experiment (Launcher Longitude dataset, $N \in \{1, 2, 4\}$ executors) confirmed that AeroSOuDA scales beneficially with executor count: the speedup at $N = 2$ was 1.867 (93.4% efficiency) and at $N = 4$ was 3.232 (80.8% efficiency). The gradual drop in efficiency from $N = 2$ to $N = 4$ is consistent with the shared-host hypothesis and validates the architecture's scaling potential.

Cross-dataset consistency. The Launcher Longitude campaign, executed with a different analytical scenario over a larger independent dataset (2.29M rows), achieved a higher speedup ($3.14\times$ at $N = 4$) and parallel efficiency (78.6%) than the Quick-Reaction campaign ($2.86\times$, 71.4%). This increase is consistent with Amdahl's law: the larger

dataset raises the parallelisable fraction of each job, reducing the relative weight of fixed serial overheads and improving the efficiency ceiling. Across both campaigns, the speedup ranged from 2.86× to 3.23×, confirming the robustness of the distributed pipeline across different input volumes and analytical configurations.

Remote communication overhead. The SSH round-trip overhead introduced by the remote client was 0.35 ± 0.03 s for the Quick-Reaction parallel mode and 0.75 ± 0.32 s for the Launcher Longitude parallel mode, less than 1% of total job time in both cases. This confirms that the remote-execution model is practical for the dataset sizes considered in this work.

Overall, the experimental results demonstrate that AeroSOuDA successfully reduces post-processing wall-clock time by 65–68% relative to the serial baseline, with the measured gain increasing with dataset size, while keeping the analyst interface simple and the deployment infrastructure standard.

7.3 Limitations

The following limitations should be considered when interpreting the results of this work.

- Single-host deployment. All benchmark experiments were conducted with Spark and HDFS containers running on the same physical machine. The data-locality benefits of HDFS, a primary architectural advantage, cannot be fully realised in this configuration. Measurements on a genuine multi-node cluster are needed to characterise the system's production-scale performance.

- Limited dataset coverage. The evaluation used two datasets: one 1.3 GB Quick-Reaction batch and one 1.5 GB Launcher Longitude batch. Smaller and larger datasets, as well as datasets with substantially different entity schemas and payload densities, may exhibit different throughput and partition-efficiency profiles.

- Integration with ASA. The current implementation operates on pre-exported CSV files produced by the ASA's Asapy library. Full end-to-end integration, in which the ASA orchestrator automatically routes simulation outputs to the AeroSOuDA cluster after each batch, has not yet been implemented.

- Metric and aggregate function coverage. The implemented processor supports Pearson correlation, mean, count, and scatter-only aggregation types. Extending the system to richer analytical functions (e.g., survival analysis, multi-variate regression) requires additions to the scenario schema and the aggregator module.

- Scenario YAML restricted to entity-state fields. The YAML scenario configuration currently allows the analyst to specify only quantities that appear as entity state fields inside the payload column of the simulation CSV. Unfortunately, not all variables are registered in the CSV simulation result files. For instance, a variable that controls the time the fighters need to fly over a specific point of the map could not be analysed throught AeroSOuDA currently, since this information is not included in said CSV files. A solution could be integrating a file along the CSV listing those variables' results separately, but that is not included in this work's scope, as it lies off the A&A stage of the simulation pipeline.

7.4 Future Work

Several directions for future work follow directly from the limitations identified above.

- Multi-node cluster evaluation. Deploying the AeroSOuDA cluster across physically separate machines would allow the data-locality principle to be fully exercised. A multi-node setup would also enable a more accurate Amdahl's law calibration by eliminating shared-host contention.

- Integration into the ASA workflow. Connecting AeroSOuDA directly to the ASA batch execution pipeline would remove the current manual CSV upload step. A webhook or queue-based trigger could initiate the distributed post-processing automatically as each simulation run completes, enabling near-real-time analytical feedback.

- Batch metadata file for alias-aware analysis. The most direct solution to the YAML limitation described in Section 7.3 is to extend the AeroSOuDA pipeline with a batch metadata file, a lightweight JSON or YAML document generated alongside the simulation CSV, containing one entry per experiment with the actual alias values used in that run. The CSV processor would then accept this file as an optional second input, resolve alias values by experiment identifier, and expose them as first-class x-variable sources in the scenario YAML, analogous to how `field_path: "frame"` is today a special-cased non-payload value. This extension would enable any variable to be correlated directly with any entity metric, without changing the structure of the simulation CSV file itself.

- Broader analytical coverage. Extending the scenario schema to support multi-variable correlations, time-series metrics, and survival probabilities would increase the system's utility for CBP analysis. Integration with the existing Asapy library

(Dantas *et al.*, 2024) could leverage its established statistical routines while adding the distributed execution layer.

- Evaluation under diverse workloads. The scenarios evaluated in this work used the same 1.3 GB production-scale input file. Benchmarks over other simulation scenarios with different entity schemas and data volumes would provide a more complete characterisation of the system's overhead profile under varied workloads.
- Security hardening. The current SSH configuration uses paramiko's `AutoAddPolicy`, acceptable for a research prototype but not for production use. A deployment-ready version should enforce host-key verification and restrict cluster access to known client addresses.

7.5 Final Remarks

This work demonstrated that the analytical bottleneck in scenario-based aerospace simulation, the phase between raw simulation output and decision-ready analytical products, can be substantially reduced through a principled application of distributed computing. The AeroSOuDA architecture is not a new simulation engine: it is a post-processing infrastructure that sits between the existing ASA workflow and the analyst's workspace, transforming large batches of raw simulation records into compact, meaningful results without burdening the analyst's machine.

The measured 2.85× speedup, achieved on conservative single-host hardware, provides empirical evidence that the MapReduce paradigm is well suited to the independent, parallel structure of simulation batch post-processing. The simplicity of the analyst interface a guided CLI requiring no Spark or HDFS expertise, further supports the system's practical utility in an operational military planning context.

The work positions itself within a broader research gap identified in the related-work survey (Section 2.7): few systems target distributed post-processing of already-persisted simulation outputs in a scenario-based aerospace-defense environment with a user-accessible, Python-centric workflow. AeroSOuDA addresses that gap with a modular, container-based implementation that can be extended, adapted, and eventually integrated into the production ASA ecosystem.

References

AMAZON WEB SERVICES. **Optimize user-defined functions, AWS Prescriptive Guidance**. 2023. Available at: https://docs.aws.amazon.com/prescriptive-guidance/latest/tuning-aws-glue-for-apache-spark/optimize-user-defined-functions.html. Accessed on: 24 Oct. 2026.

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. *In:* PROCEEDINGS of the AFIPS Spring Joint Computer Conference. [*S.l.: s.n.*], 1967. p. 483–485. DOI: 10.1145/1465482.1465560.

APACHE SOFTWARE FOUNDATION. **Tuning Apache Spark: JVM Garbage Collection and Memory**. [*S.l.: s.n.*], 2024. https://spark.apache.org/docs/latest/tuning.html. Available at: https://spark.apache.org/docs/latest/tuning.html. Accessed on: 1 Nov. 2026.

APACHE SPARK PROJECT. **PySpark Documentation**. [*S.l.: s.n.*], 2026. Available at: https://spark.apache.org/docs/latest/api/python/index.html. Accessed on: 4 June 2026.

APACHE SPARK PROJECT. **RDD Programming Guide**. [*S.l.: s.n.*], 2024. Available at: https://spark.apache.org/docs/latest/rdd-programming-guide.html. Accessed on: 11 Nov. 2026.

ARMBRUST, M.; XIN, R. S.; LIAN, C.; HUAI, Y.; LIU, D.; BRADLEY, J. K.; MENG, X.; KAFTAN, T.; FRANKLIN, M. J.; GHODSI, A.; ZAHARIA, M. Spark SQL: Relational Data Processing in Spark. *In:* PROCEEDINGS of the 2015 ACM SIGMOD International Conference on Management of Data. [*S.l.*]: ACM, 2015. p. 1383–1394. DOI: 10.1145/2723372.2742797.

BEN-KIKI, O.; EVANS, C.; INGERSON, B. **YAML Ain't Markup Language (YAML) Version 1.1**. [*S.l.*], 2009. Available at: https://yaml.org/spec/1.1/. Accessed on: 4 June 2026.

DANTAS, J. P. A.; COSTA, A. N.; GOMES, V. C. F.; KUROSWISKI, A. R.; MEDEIROS, F. L. L.; GERALDO, D.; SAMERSLA, A. R.; SILVA, S. R.; CAETANO, A. F. M.; FERREIRA, Y. D.; ARANTES, A. O.; SANTOS, D. S.;

AQUINO, M. R. C. ASA: A Simulation Environment for Evaluating Military Operational Scenarios. *In:* ARABNIA, H. R.; DELIGIANNIDIS, L.; SHENAVARMASOULEH, F.; AMIRIAN, S.; GHAREH MOHAMMADI, F. (ed.). **Computational Science and Computational Intelligence**. [*S.l.*]: Springer Nature Switzerland, 2025. p. 203–215. ISBN 978-3-031-99586-6. DOI: 10.1007/978-3-031-99586-6_17.

DANTAS, J. P. A.; SILVA, S. R.; GOMES, V. C. F.; COSTA, A. N.; SAMERSLA, A. R.; GERALDO, D.; MAXIMO, M. R. O. A.; YONEYAMA, T. AsaPy: A Python Library for Aerospace Simulation Analysis. *In:* PROCEEDINGS of the 38th ACM SIGSIM Conference on Principles of Advanced Discrete Simulation. Atlanta, GA, USA: Association for Computing Machinery, 2024. (SIGSIM-PADS '24), p. 15–24. ISBN 9798400703638. DOI: 10.1145/3615979.3656063. Available at: https://doi.org/10.1145/3615979.3656063.

DAVIS, P. K. **Analytic Architecture for Capabilities-Based Planning, Mission-System Analysis, and Transformation**. Santa Monica, CA, 2002. Available at: https://www.rand.org/pubs/monograph_reports/MR1513.html. Accessed on: 4 June 2026.

DEAN, J.; GHEMAWAT, S. MapReduce: Simplified Data Processing on Large Clusters. *In:* PROCEEDINGS of the 6th USENIX Symposium on Operating Systems Design and Implementation. [*S.l.*]: USENIX, 2004. p. 137–150.

DEELMAN, E.; VAHI, K.; JUVE, G.; RYNGE, M.; CALLAGHAN, S.; MAECHLING, P. J.; MAYANI, R.; CHEN, W.; FERREIRA DA SILVA, R.; LIVNY, M.; WENGER, K. Pegasus, a Workflow Management System for Science Automation. **Future Generation Computer Systems**, v. 46, p. 17–35, 2015. DOI: 10.1016/j.future.2014.10.008.

DOCKER, INC. **Docker Compose documentation**. [*S.l.: s.n.*], 2024. Available at: https://docs.docker.com/compose/. Accessed on: 11 Nov. 2026.

EGFJORD, K. F.-H.; SUND, K. J. A modified Delphi method to elicit and compare perceptions of industry trends. **MethodsX**, v. 7, p. 101081, 2020. ISSN 2215-0161. DOI: https://doi.org/10.1016/j.mex.2020.101081. Available at: https://www.sciencedirect.com/science/article/pii/S2215016120303010.

EKANAYAKE, J.; LI, H.; ZHANG, B.; GUNARATHNE, T.; BAE, S.-H.; CHO, J.; FOX, G. Twister: a runtime for iterative mapreduce. *In:* ACM. PROCEEDINGS of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC). [*S.l.: s.n.*], 2010. p. 810–818. DOI: 10.1145/1851476.1851593.

FOSTER, I.; KESSELMAN, C. **The Grid 2: Blueprint for a New Computing Infrastructure**. 2nd. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2003. ISBN 1-55860-933-4.

FRIESEN, B.; WILLIAMS, S.; WU, K.; PRABHAT; HAMANN, B.; BETHEL, W.; LUKIĆ, Z.; WOILLEZ, J.; WEBER, G. H. In situ and in-transit analysis of cosmological simulations. **Computational Astrophysics and Cosmology**, v. 3, n. 4, p. 1–18, 2016. DOI: 10.1186/s40668-016-0017-2.

FUJIMOTO, R. M. Research challenges in parallel and distributed simulation. **ACM Transactions on Modeling and Computer Simulation**, v. 26, n. 4, 22:1–22:29, 2016. DOI: 10.1145/2866577.

GHEMAWAT, S.; GOBIOFF, H.; LEUNG, S. The Google File System. *In:* PROCEEDINGS of the Nineteenth ACM Symposium on Operating Systems Principles. [*S.l.*]: ACM, 2003. p. 29–43. DOI: 10.1145/945445.945450.

HENNESSY, J. L.; PATTERSON, D. A. A New Golden Age for Computer Architecture. **Communications of the ACM**, ACM New York, NY, USA, v. 62, n. 2, p. 48–60, 2019. DOI: 10.1145/3282307.

HODICKÝ, J.; PROCHÁZKA, D.; BAXA, F.; MELICHAR, J.; KREJČÍK, M.; KŘÍŽEK, P.; STODOLA, P.; DROZD, J. Computer Assisted Wargame for Military Capability-Based Planning. **Entropy**, v. 22, n. 8, p. 861, 2020. DOI: 10.3390/e22080861.

HODSON, D. D.; HILL, R. R. The art and science of live, virtual, and constructive simulation for test and analysis. **Journal of Defense Modeling and Simulation: Applications, Methodology, Technology**, v. 11, n. 2, p. 77–89, 2014. DOI: 10.1177/1548512913506620.

LAGARES, C.; RIVERA, W.; ARAYA, G. Scalable Post-Processing of Large-Scale Numerical Simulations of Turbulent Fluid Flows. **Symmetry**, v. 14, n. 4, p. 823, 2022. DOI: 10.3390/sym14040823.

LI, Y.; ZHANG, X.; SRINATH, A.; GETMAN, R. B.; NGO, L. B. Combining HPC and Big Data Infrastructures in Large-Scale Post-Processing of Simulation Data: A Case Study. *In:* PROCEEDINGS of the Practice and Experience in Advanced Research Computing Conference. [*S.l.*]: ACM, 2018. p. 1–7. DOI: 10.1145/3219104.3229279.

MCGUGAN, W. **Rich: Rich text and beautiful formatting in the terminal**. [*S.l.: s.n.*], 2024. Available at: https://github.com/Textualize/rich. Accessed on: 4 June 2026.

MCKINNEY, W. Apache Arrow and the "10 Things I Hate About Pandas". *In:* PYDATA DC. [*S.l.: s.n.*], 2016. Available at: https://wesmckinney.com/blog/apache-arrow-pandas-internals/. Accessed on: 24 Oct. 2026.

MERKEL, D. Docker: Lightweight Linux Containers for Consistent Development and Deployment. **Linux Journal**, Belltown Media, v. 2014, n. 239, 2014.

MONTGOMERY, D. C.; RUNGER, G. C. **Applied Statistics and Probability for Engineers**. 6th. Hoboken, NJ, USA: John Wiley & Sons, 2014. ISBN 978-1-118-53971-2.

MORRISSEY, J. P.; TOTOO, P.; HANLEY, K. J.; PAPANICOLOPULOS, S.-A.; OOI, J. Y.; CORES GONZALEZ, I.; RAFFIN, B.; MOSTAJABODAVEH, S.; GIERLINGER, T. Post-processing and visualization of large-scale DEM simulation data with the open-source VELaSSCo platform. **Simulation: Transactions of the Society for Modeling and Simulation International**, v. 96, n. 7, p. 567–581, 2020. DOI: 10.1177/0037549720906465.

MÜLLER, M.; BERNARD, L.; KADNER, D. Moving code - Sharing geoprocessing logic on the Web. **ISPRS Journal of Photogrammetry and Remote Sensing**, v. 83, p. 193–203, 2013. ISSN 0924-2716. DOI: https://doi.org/10.1016/j.isprsjprs.2013.02.011. Available at: https://www.sciencedirect.com/science/article/pii/S0924271613000543.

OPENBSD. **OpenSSH**. [*S.l.: s.n.*], 2024a. Available at: https://www.openssh.com/. Accessed on: 11 Nov. 2026.

OPENBSD. **sftp(1) manual page**. [*S.l.: s.n.*], 2024b. Available at: https://man.openbsd.org/sftp. Accessed on: 11 Nov. 2026.

ORACLE. **Java Virtual Machine**. [*S.l.: s.n.*], 2024. Available at: https://docs.oracle.com/en/java/javase/17/docs/specs/jvms/se17/html/. Accessed on: 11 Nov. 2026.

PARAMIKO CONTRIBUTORS. **Paramiko: SSH2 protocol library for Python**. [*S.l.: s.n.*], 2024. Available at: https://www.paramiko.org/. Accessed on: 29 Oct. 2026.

PARTRIDGE, C. **Late-Binding RPC: A Paradigm for Distributed Computation in a Gigabit Environment**. 1992. Ph.D. thesis – Harvard University, Cambridge, MA.

PEARSON, K. Notes on regression and inheritance in the case of two parents. **Proceedings of the Royal Society of London**, v. 58, p. 240–242, 1895. DOI: 10.1098/rspl.1895.0041.

REED, D. A.; DONGARRA, J. Exascale computing and big data. **Communications of the ACM**, ACM New York, NY, USA, v. 58, n. 7, p. 56–68, 2015. DOI: 10.1145/2699414.

SEVILLA, M.; NASSI, I.; IOANNIDOU, K.; BRANDT, S.; MALTZAHN, C. **A Framework for an In-depth Comparison of Scale-up and Scale-out**. [*S.l.*], 2015.

STEEN, M. v.; TANENBAUM, A. S. A Brief Introduction to Distributed Systems. **Computing**, Springer, v. 98, n. 10, p. 967–1009, 2016. DOI: 10.1007/s00607-016-0508-7.

TECHNOLOGIES, R.; CONTRIBUTORS. **questionary: A Python library for building pretty command line interfaces**. [*S.l.: s.n.*], 2024. Available at: https://github.com/tmbo/questionary. Accessed on: 4 June 2026.

VALDURIEZ, P.; MATTOSO, M.; AKBARINIA, R.; BORGES, H.; CAMATA, J. Scientific Data Analysis Using Data-Intensive Scalable Computing: the SciDISC Project. **HAL - Archives Ouvertes**, Sept. 2018. Available at: https://hal-lirmm.ccsd.cnrs.fr/lirmm-01867804.

WALDROP, M. M. The chips are down for Moore's law. **Nature**, v. 530, n. 7589, p. 144–147, 2016. DOI: 10.1038/530144a.

WIĘCKOWSKI, J.; SAŁABUN, W. Sensitivity analysis approaches in multi-criteria decision analysis: A systematic review. **Applied Soft Computing**, v. 148, p. 110915, 2023. DOI: 10.1016/j.asoc.2023.110915.

ZAHARIA, M.; CHOWDHURY, M.; DAS, T.; DAVE, A.; MA, J.; MCCAULEY, M.; FRANKLIN, M. J.; SHENKER, S.; STOICA, I. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *In:* PROCEEDINGS of the 9th USENIX Symposium on Networked Systems Design and Implementation. [*S.l.*]: USENIX, 2012. p. 15–28.

ZHENG, Q. **Distributed Metadata and Streaming Data Indexing as Scalable Filesystem Services**. 2021. PhD thesis – Carnegie Mellon University, Pittsburgh, PA, USA.